



COSMOS AI at Scale

Convolutional Neural Network

July 2026, Instructor: Meng Tang

Some slides from Yuri Boykov and Yann LeCun

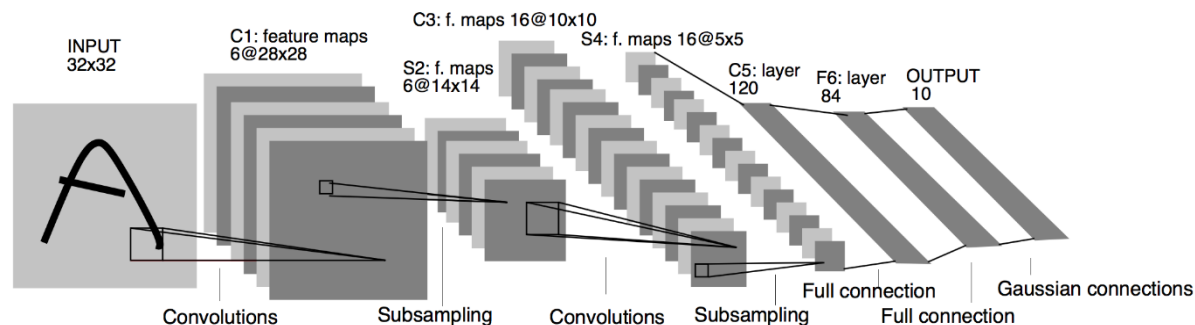
Convolutional Neural Networks (CNNs)

for image classification

- **convolutional layers**
- **pooling** (max and average)
- **fully connected layers**
- **data augmentation**
- **class activation map (CAM)**

First CNN architectures for classification

- **first CNNs (1982-89)** *Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in position*
(a.k.a. *convNets*)
K. Fukushima, S. Miyake - Pattern Recognition 1982
Handwritten digit recognition with a back-propagation network
Y. LeCun et al - NIPS 1989
- **LeNet (1998)** *Handwritten digit recognition with a back-propagation network*
Y. LeCun, L. Bottou, Y. Bengio, P. Haffner - Proc.of IEEE 1998



First CNN architectures for classification

- **first CNNs (1982-89)** *Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in position*
(a.k.a. *convNets*)
K. Fukushima, S. Miyake - Pattern Recognition 1982
Handwritten digit recognition with a back-propagation network
Y. LeCun et al - NIPS 1989
- **LeNet (1998)**



Early Work on CNNs

Fukushima (1980) – Neo-Cognitron

LeCun (1998) – Convolutional Networks (ConvNets)

- similarities to Neo-Cognitron
- success on character recognition

Other attempts at deeply layered Networks trained with backpropagation

- not much success (e.g. very slow, diffusing/vanishing gradient)

Since 2010 - significant training improvements

- various tricks (batch normalization, drop-outs, residual links, etc.)

Deep CNN architectures for classification

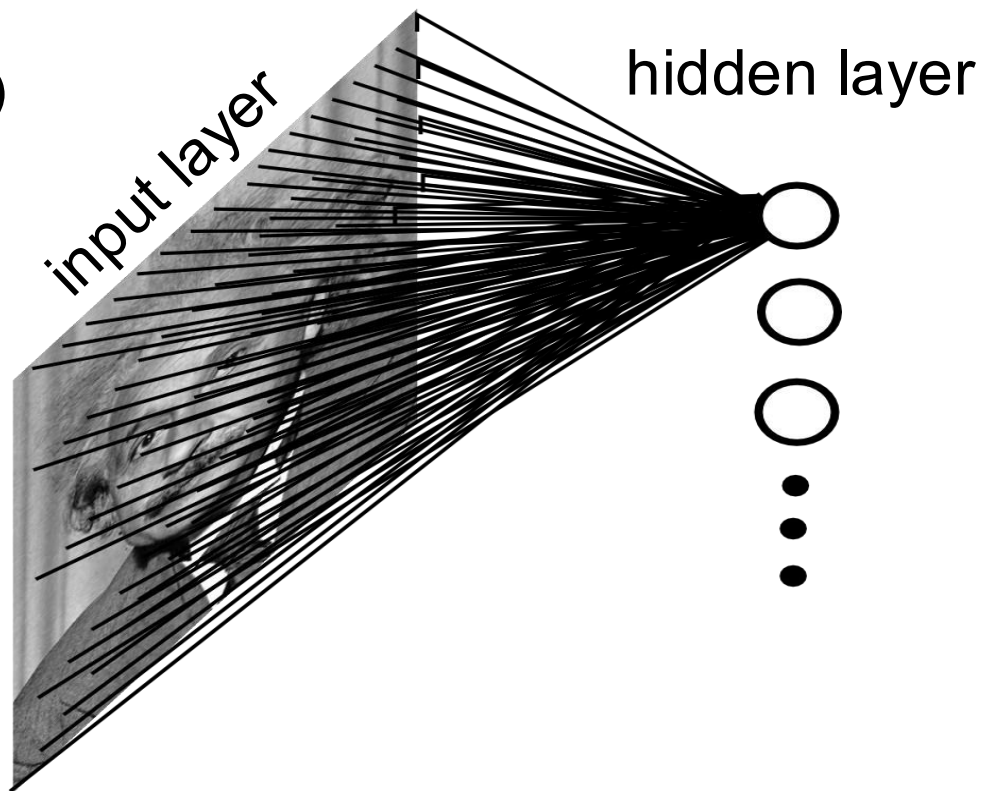
- **AlexNet** (2012) *ImageNet classification with deep convolutional neural networks*
Alex Krizhevsky, Ilya Sutskever, Geoffrey Hinton - NIPS 2012.
- **VGG** (2014) *Very Deep Convolutional Networks for Large-Scale Image Recognition*
K. Simonyan, A. Zisserman - ICLR 2015
<http://www.robots.ox.ac.uk/~vgg/practicals/cnn/index.html>
- **ResNet** (2016) *Deep residual learning for image recognition*
K. He, X. Zhang, S. Ren, J. Sun. - CVPR 2016

Convolutional Network: Motivation

Consider a **fully connected network** (most weights $W[i,j] \neq 0$)

Example: 200 by 200 image,
 4×10^4 connections to one
hidden unit

For 10^5 hidden units $\rightarrow 4 \times 10^9$
connections



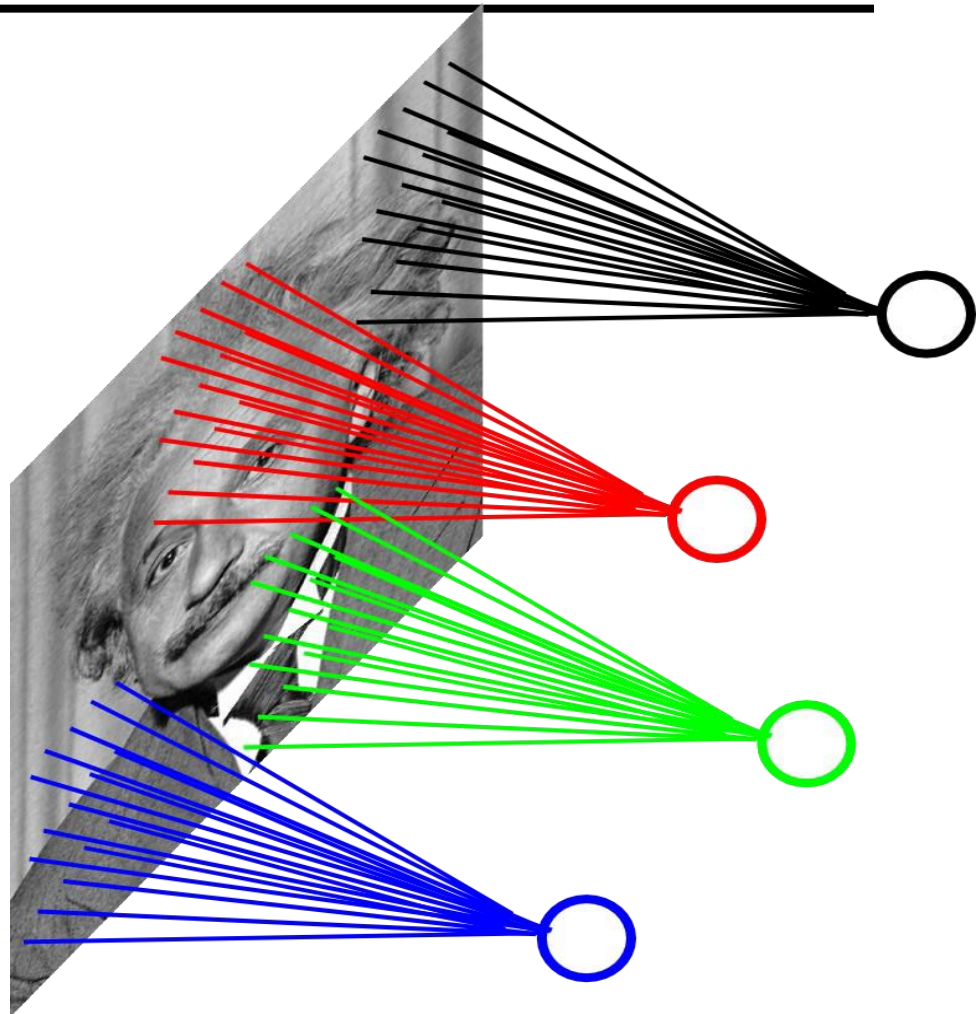
Convolutional Network: Motivation

Connect only pixels in a local patch, say 10×10

For 200 by 200 image, 10^2 connections to one hidden unit

For 10^5 hidden units $\rightarrow 10^7$ connections

- contrast with 4×10^9 for fully connected layer
- factor of 400 decrease



Invariance to Translation

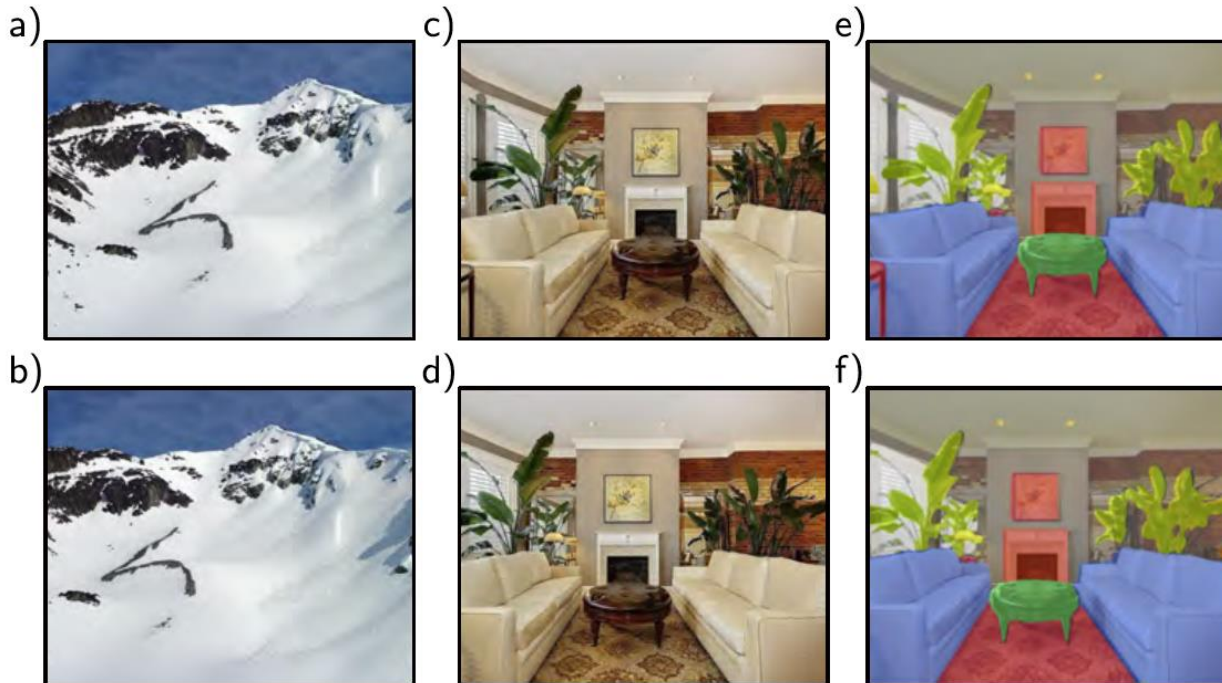
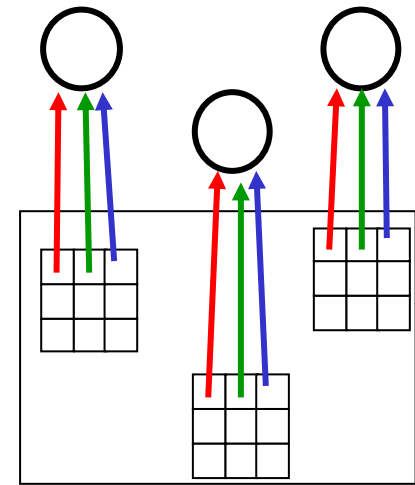
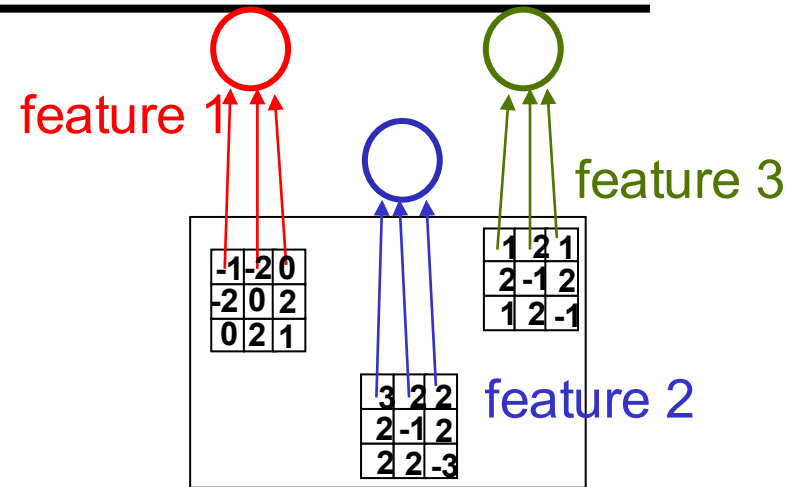


Figure 10.1 Invariance and equivariance for translation. a–b) In image classification, the goal is to categorize both images as “mountain” regardless of the horizontal shift that has occurred. In other words, we require the network prediction to be invariant to translation. c,e) The goal of semantic segmentation is to associate a label with each pixel. d,f) When the input image is translated, we want the output (colored overlay) to translate in the same way. In other words, we require the output to be equivariant with respect to translation. Panels c–f) adapted from Bousselham et al. (2021).

Invariance to Translation

- Intuitively, each neuron learns a good feature (a filter) in one particular location
- Idea: make all neurons detect the **same feature at different positions**
 - i.e. **share parameters** (network weights) across different locations



red connections have equal weight
green connections have equal weight
blue connections have equal weight

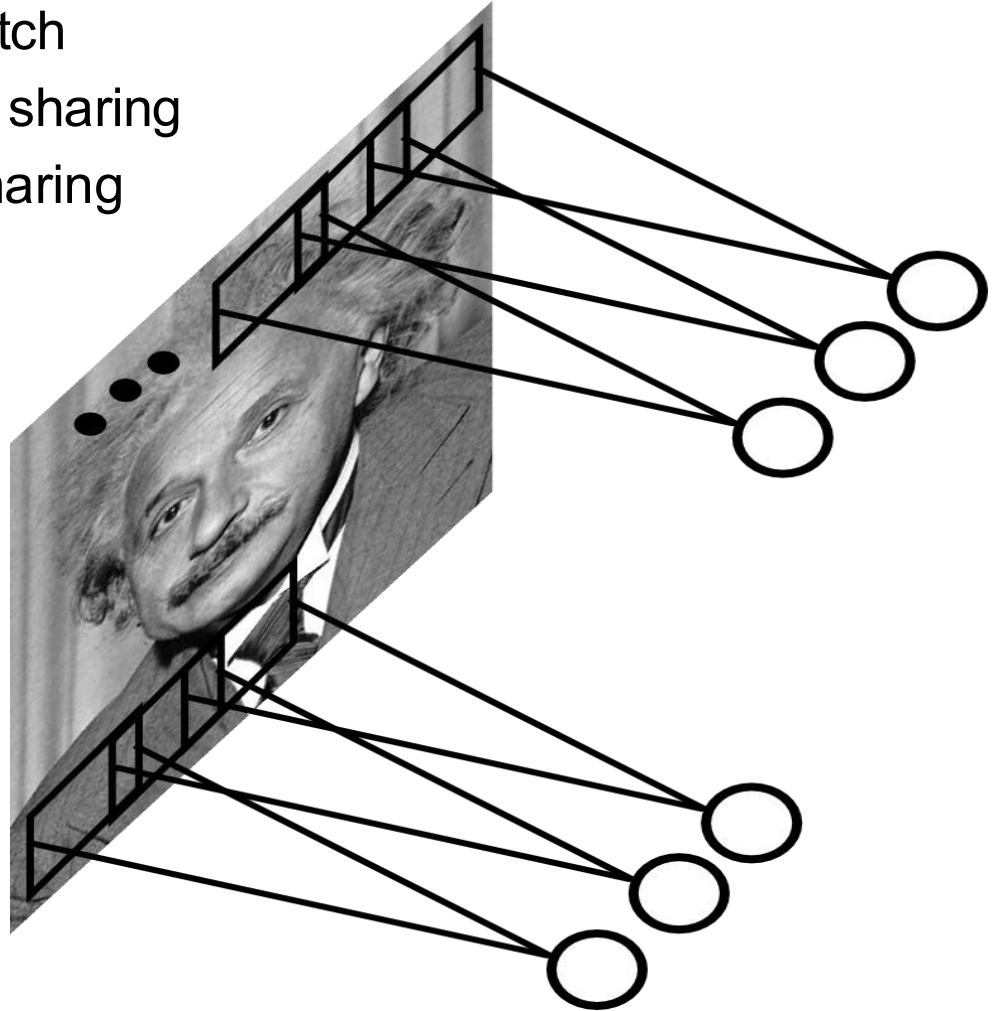
ConvNets: Weight Sharing

Much fewer parameters to learn

For 10^5 hidden units and 10×10 patch

- 10^7 parameters to learn without sharing
- 10^2 parameters to learn with sharing

Does not depend
on the number of
hidden units



2D Convolution/Filtering

A 2D image $f[i,j]$ can be filtered by a **2D kernel** $h[u,v]$ to produce an output image $g[i,j]$:

$$g[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u, v] \cdot f[i + u, j + v]$$

This is called a **convolution** operation and written:

$$g = h \circ f$$

h is called “**kernel**” or “**mask**” or “**filter**” which representing a given “window function”

2D Filtering for Noise Reduction

- ❑ Common types of noise:
 - ❑ **Salt and pepper noise:** random occurrences of black and white pixels
 - ❑ **Impulse noise:** random occurrences of white pixels
 - ❑ **Gaussian noise:** variations in intensity drawn from a Gaussian normal distribution



Original



Salt and pepper noise



Impulse noise



Gaussian noise

Mean filtering

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[x, y]$

		10							

$g[x, y]$

Mean filtering

[illegible]
$$f[x, y]$$

A 10x10 grid is shown. The grid is composed of 10 columns and 10 rows. A red square is located at the intersection of the 5th column and the 6th row, containing the number 80. A black square is located at the intersection of the 3rd column and the 3rd row, containing the number 10.

$$g[x, y]$$

Mean filtering

side effect of mean filtering: **blurring**

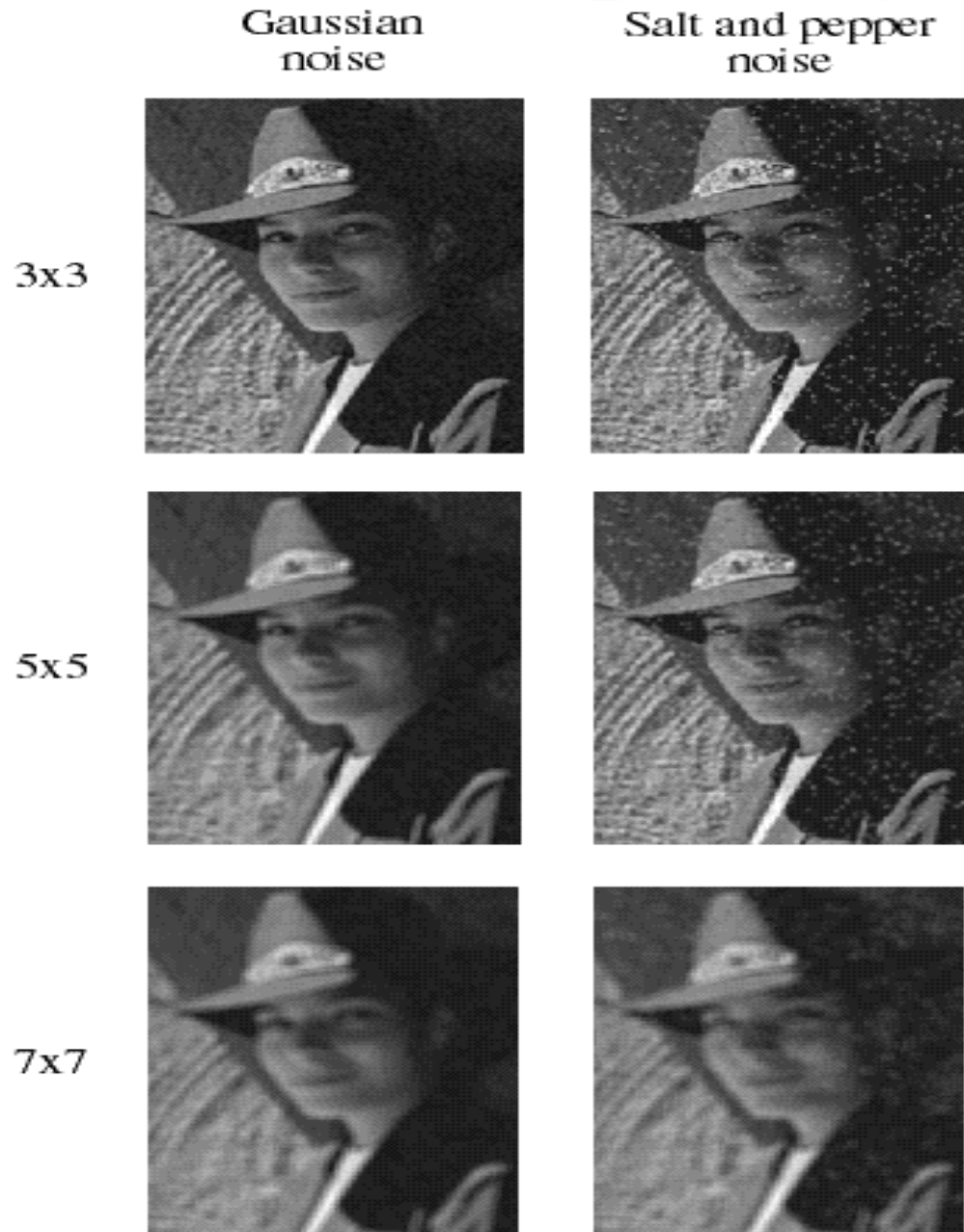
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$f[x, y]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$g[x, y]$

Mean filtering



Mean kernel

□ What's the kernel for a 3x3 mean filter?

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

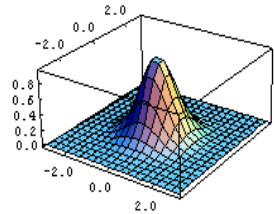
$$\frac{1}{9} \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Gaussian filtering

□ A Gaussian kernel gives less weight to pixels further from the center

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$\frac{1}{16} \cdot \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



discrete approximation of
a Gaussian (density) function

$$h(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{\sigma^2}}$$

Median filter

- ☐ A **Median Filter** operates over a window by selecting the median intensity in the window.
- ☐ What advantage does a median filter have over a mean filter?
- ☐ Is a median filter a kind of convolution?
 - ☐ - No, median filter is non-linear

Comparison: salt and pepper noise

3x3



5x5

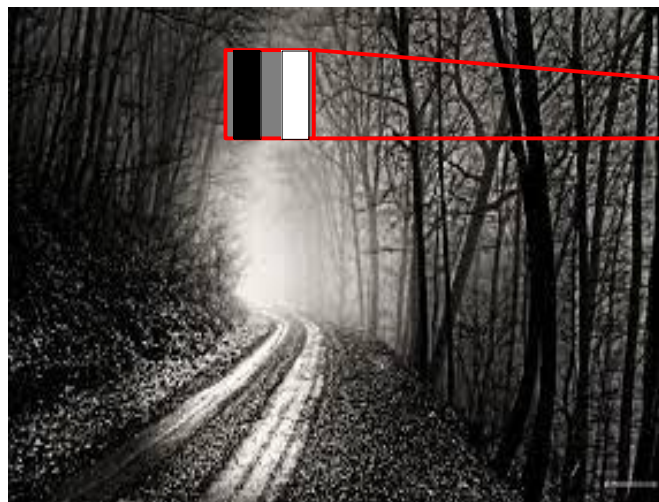


7x7

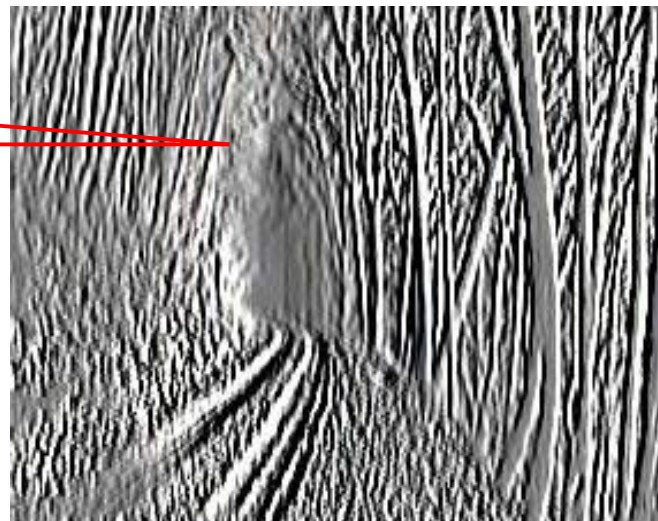


Filtering via Convolution

filtering with convolution for feature extraction



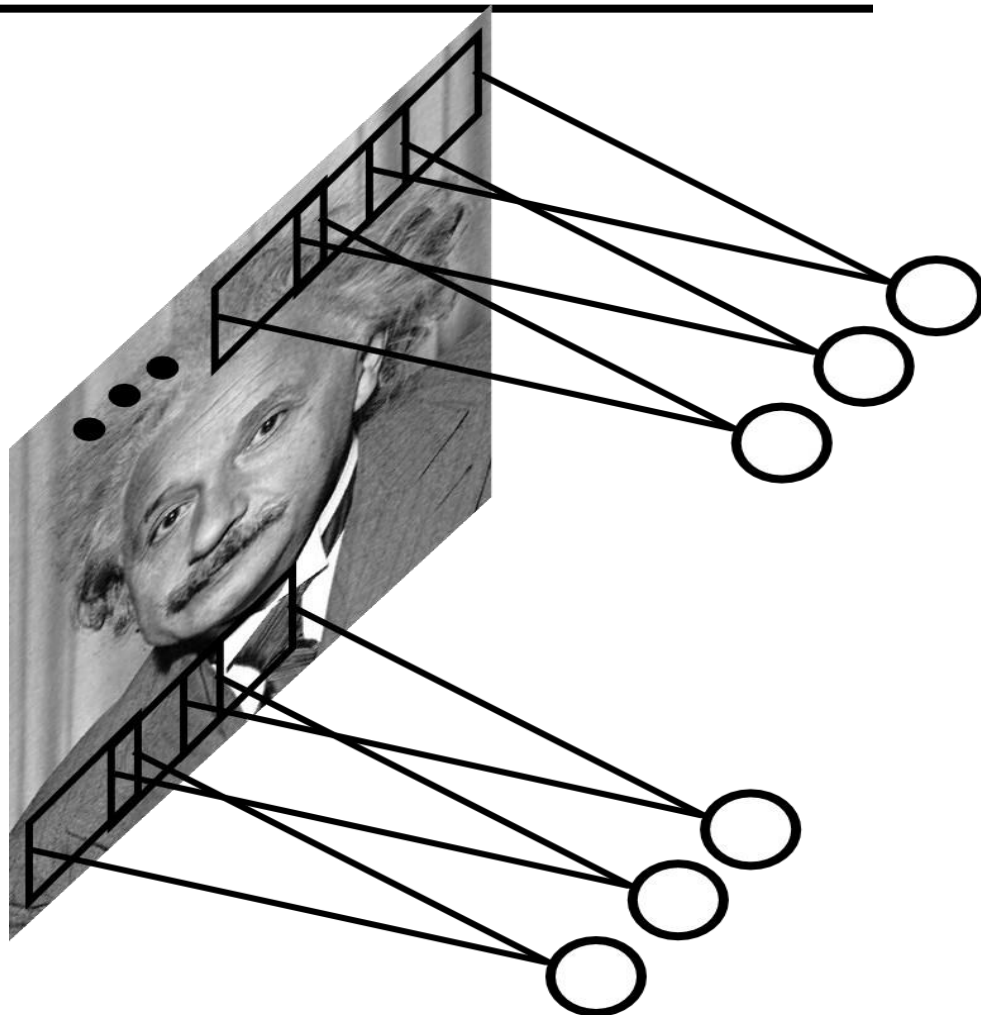
$$\begin{matrix} * & \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} & = \end{matrix}$$



Convolutional Layer

Same as convolution with
some fixed filter

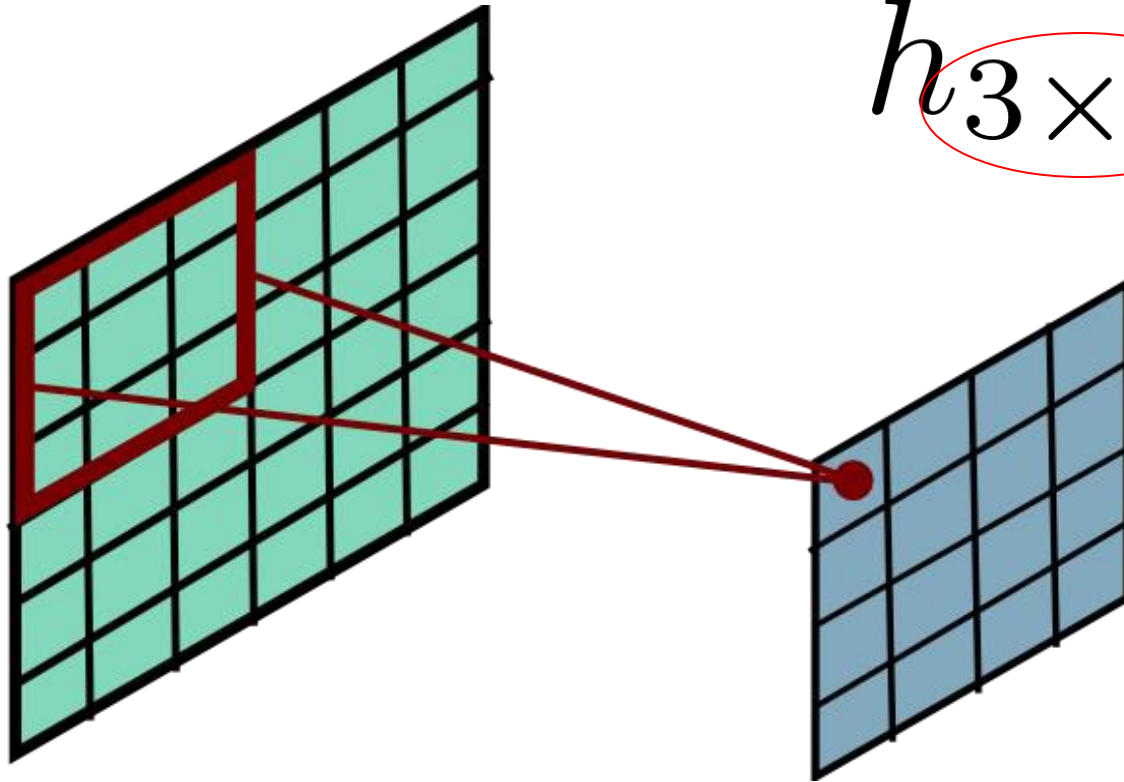
But here the filter
parameters
will be learned



Convolutional Layer

convolution kernel

h 3×3 size



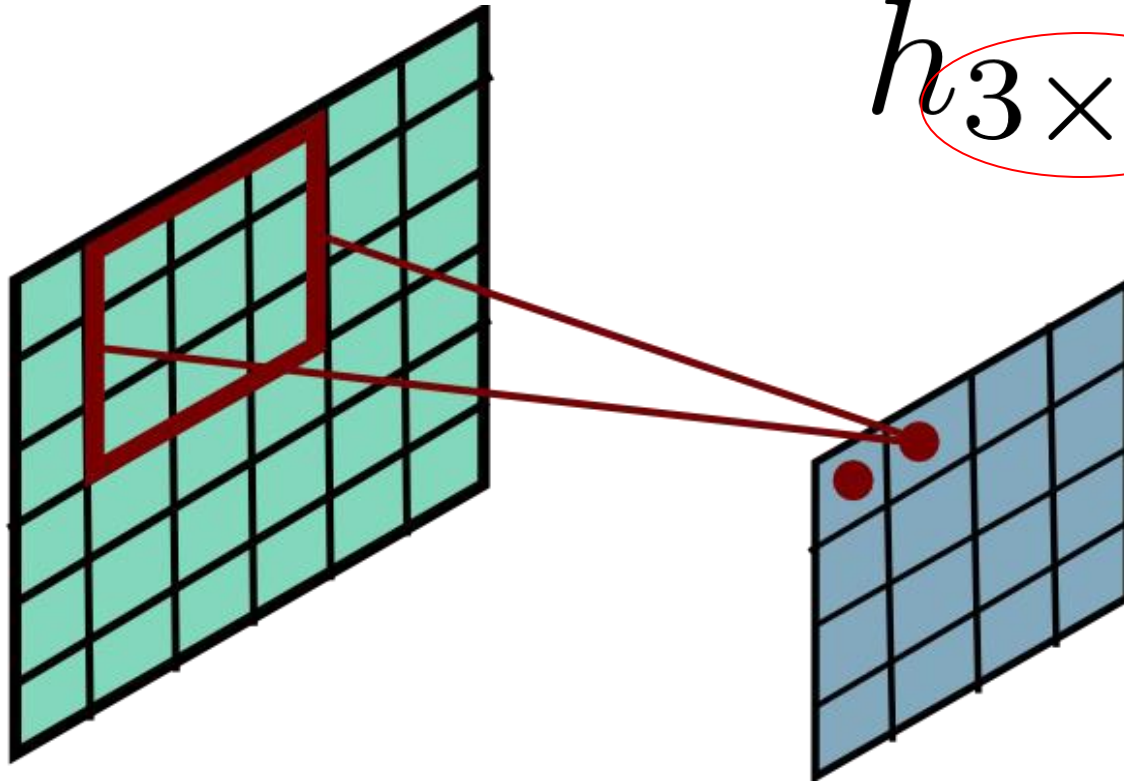
input

output

Convolutional Layer

convolution kernel

h 3×3 size



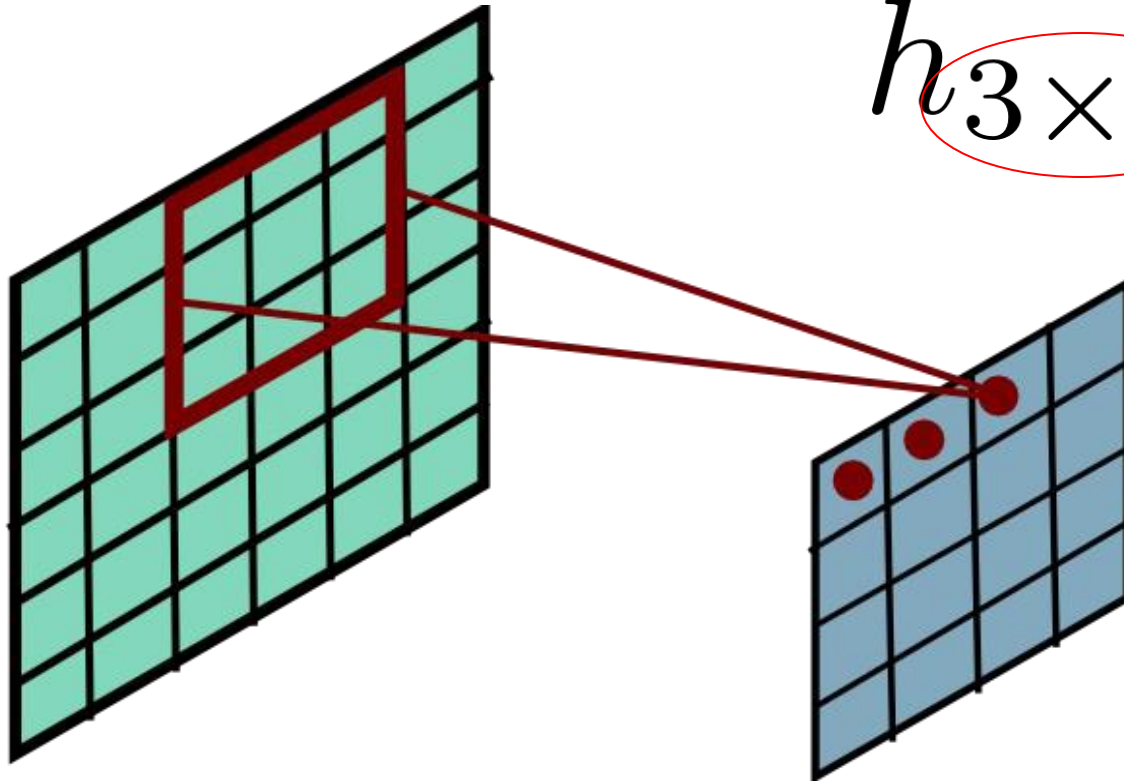
input

output

Convolutional Layer

convolution kernel

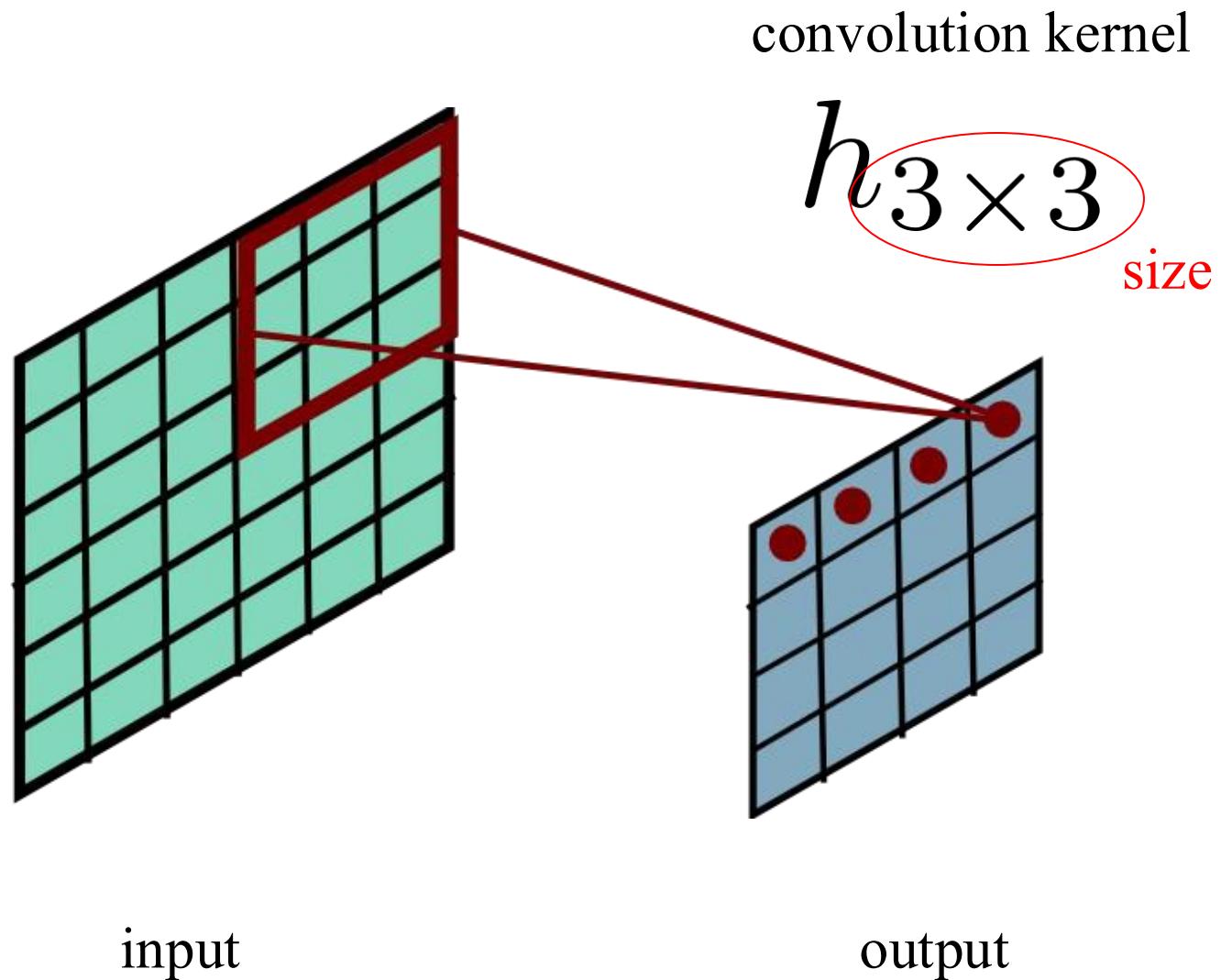
h 3×3 size



input

output

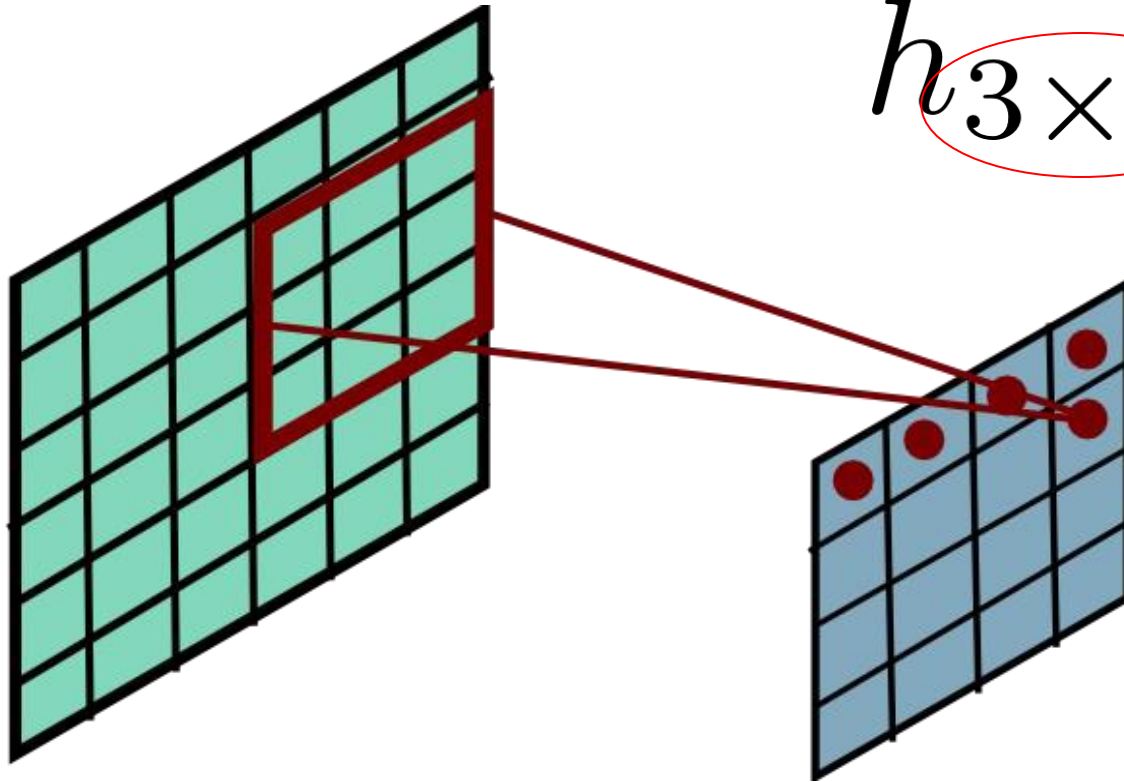
Convolutional Layer



Convolutional Layer

convolution kernel

h 3×3 size



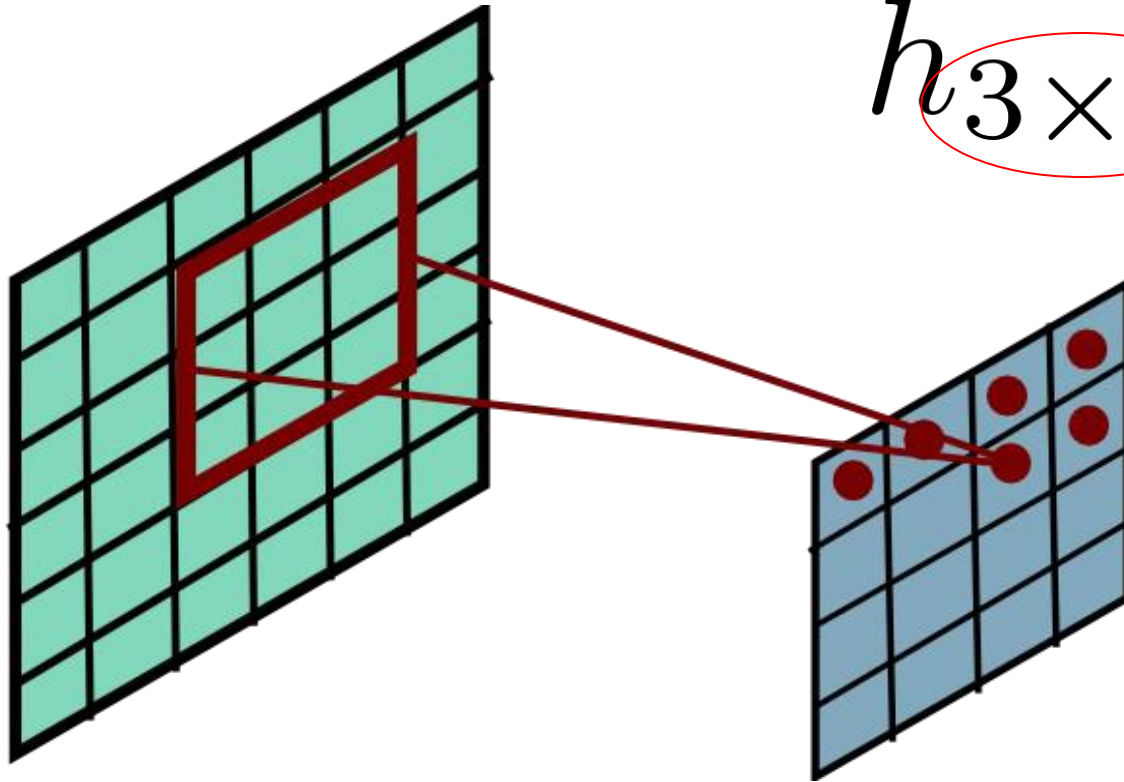
input

output

Convolutional Layer

convolution kernel

h 3×3 size



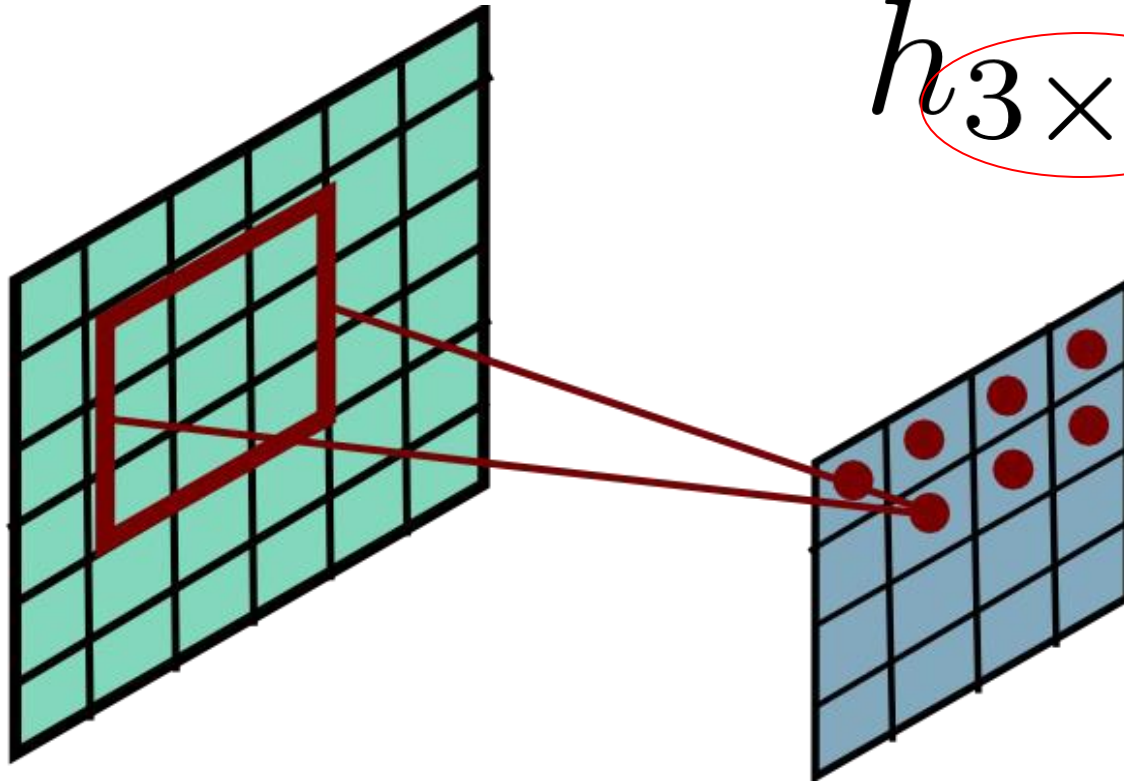
input

output

Convolutional Layer

convolution kernel

h 3×3 size



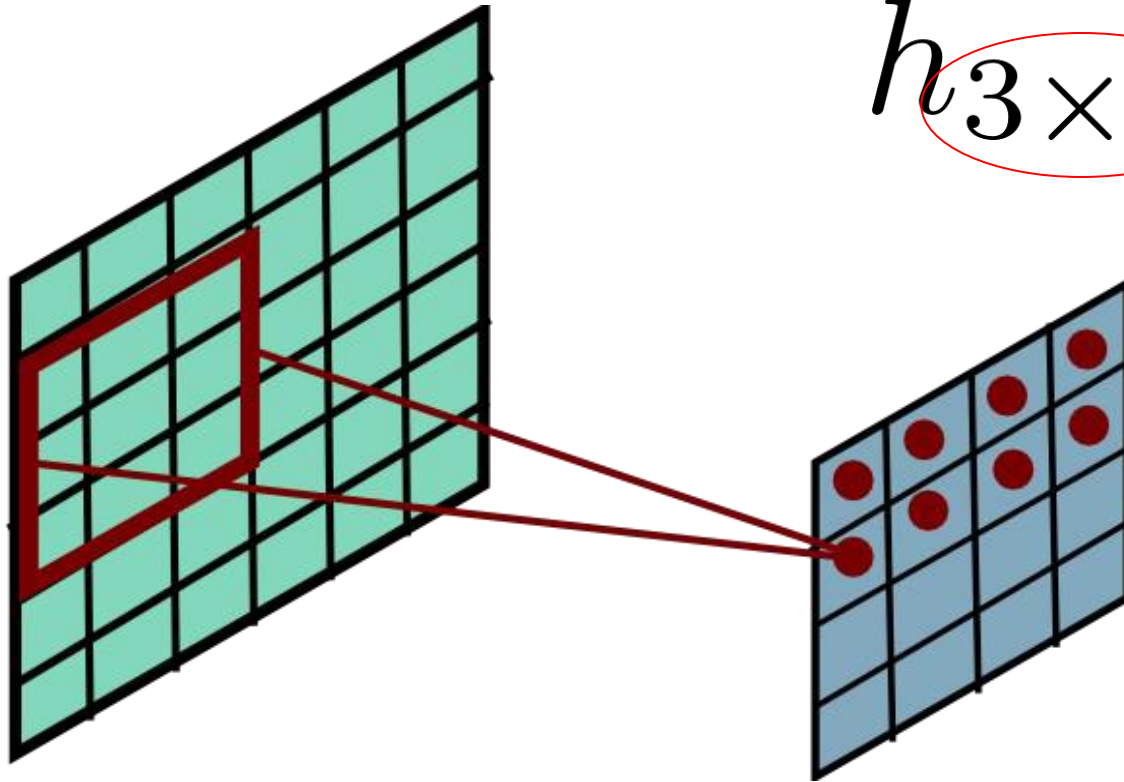
input

output

Convolutional Layer

convolution kernel

h 3×3 size



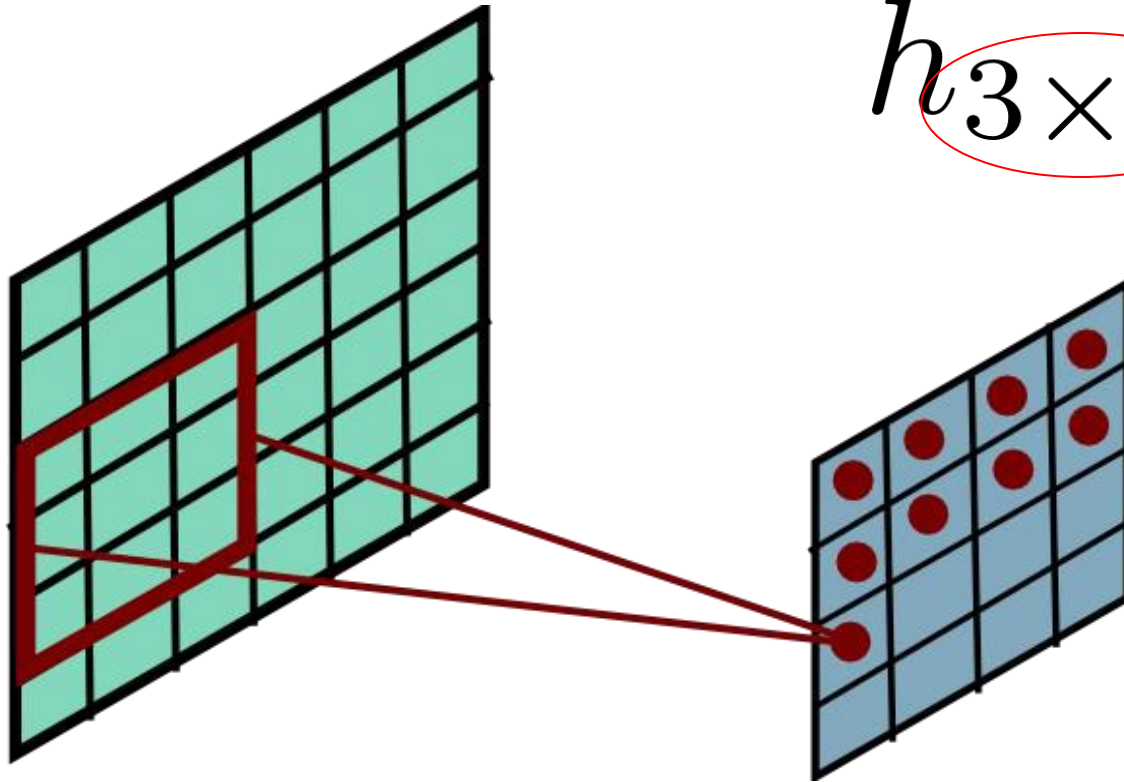
input

output

Convolutional Layer

convolution kernel

h 3×3 size



input

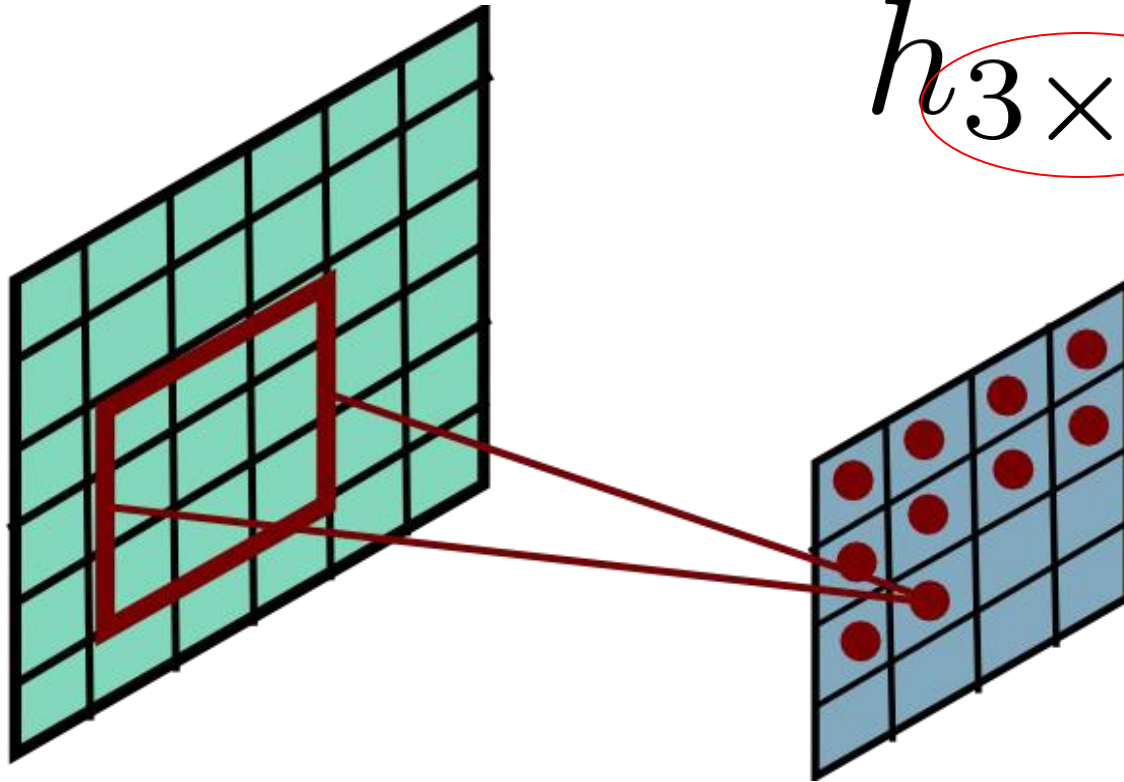
output

Convolutional Layer

convolution kernel

$$h_{3 \times 3}$$

size



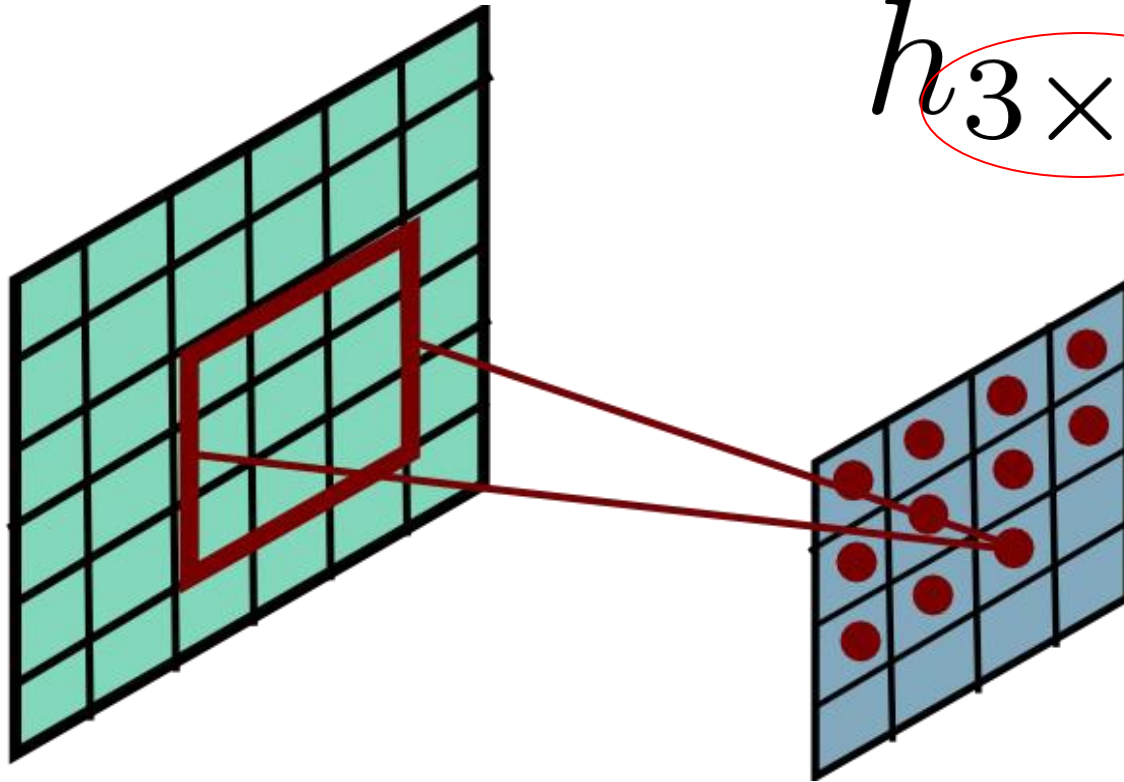
input

output

Convolutional Layer

convolution kernel

h 3×3 size



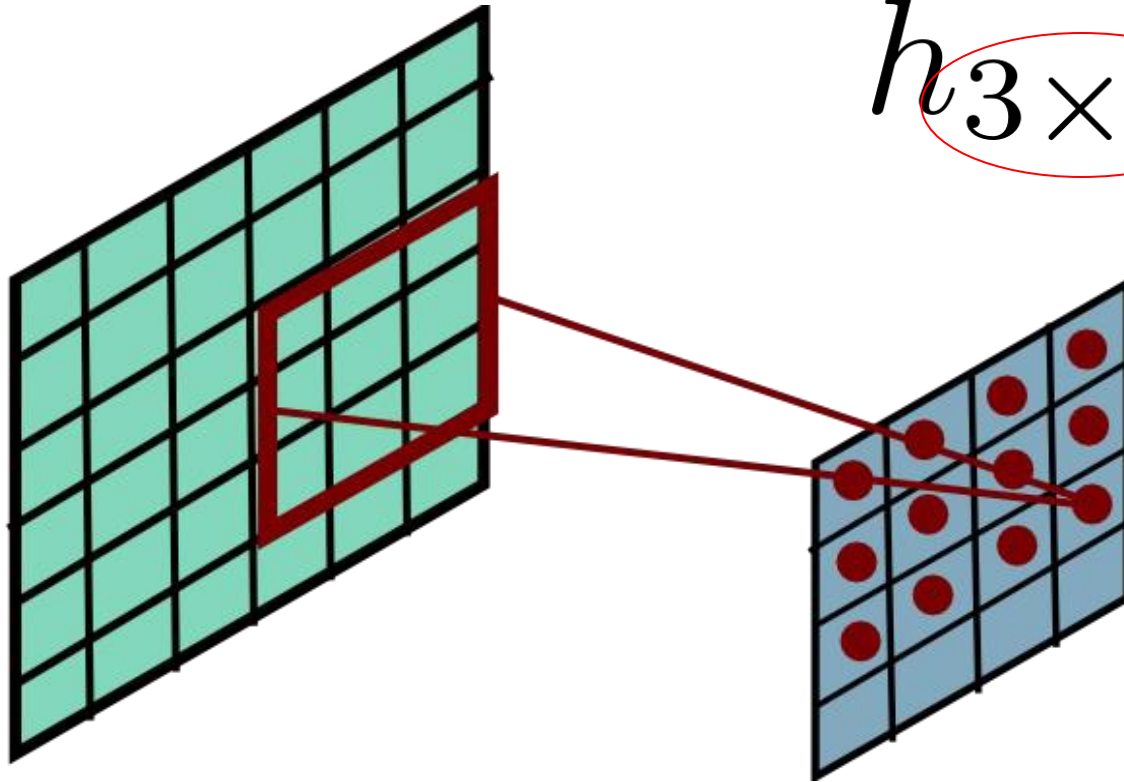
input

output

Convolutional Layer

convolution kernel

h 3×3 size



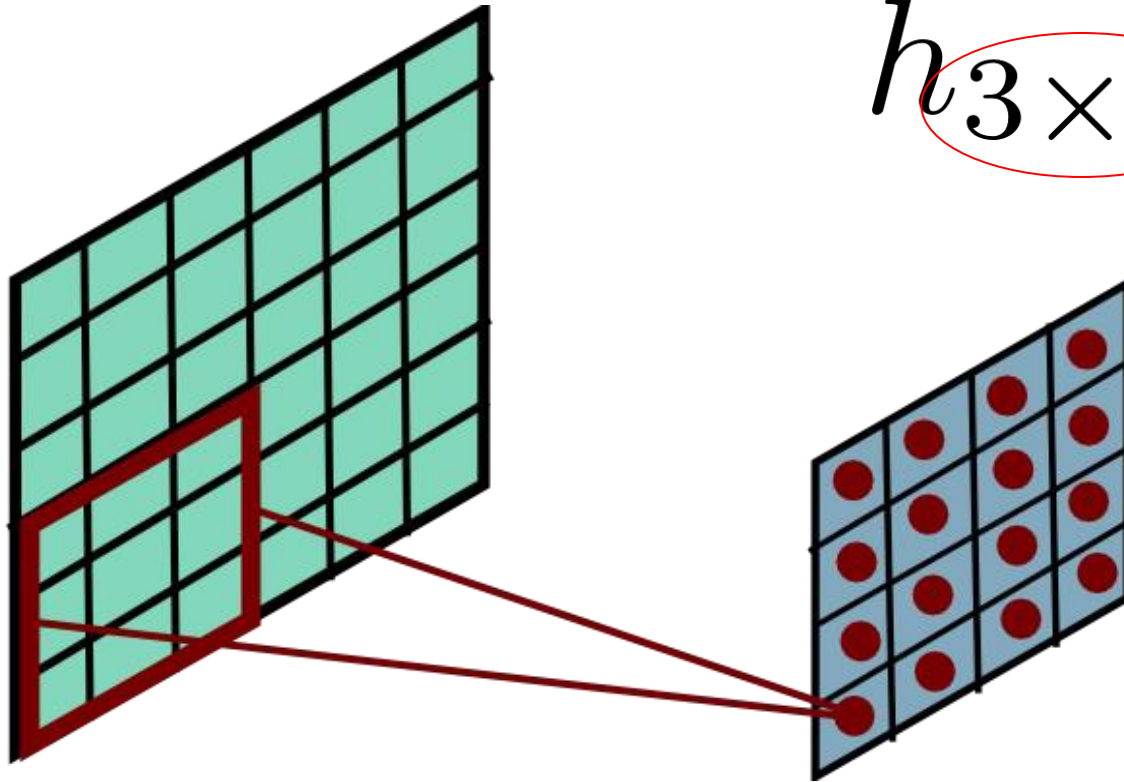
input

output

Convolutional Layer

convolution kernel

h 3×3 size

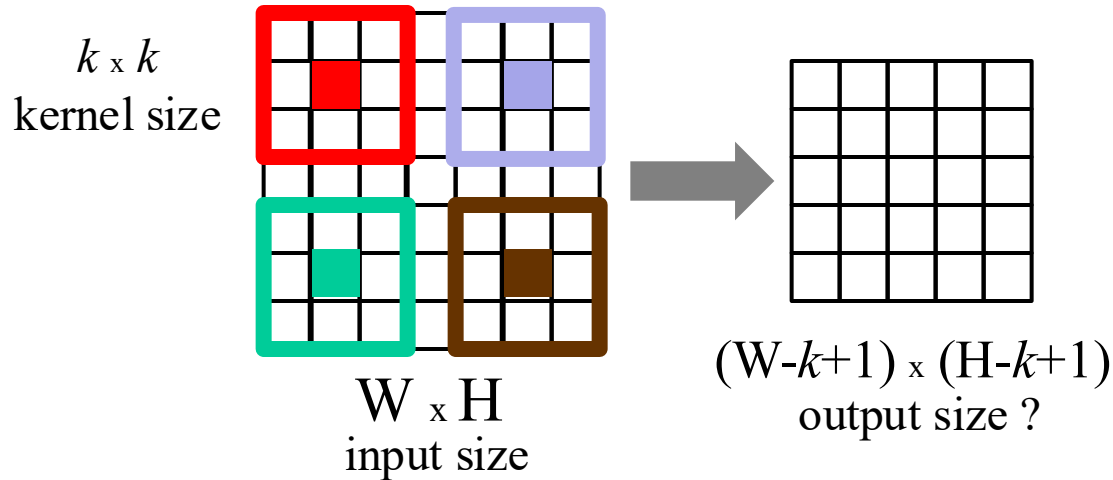


input

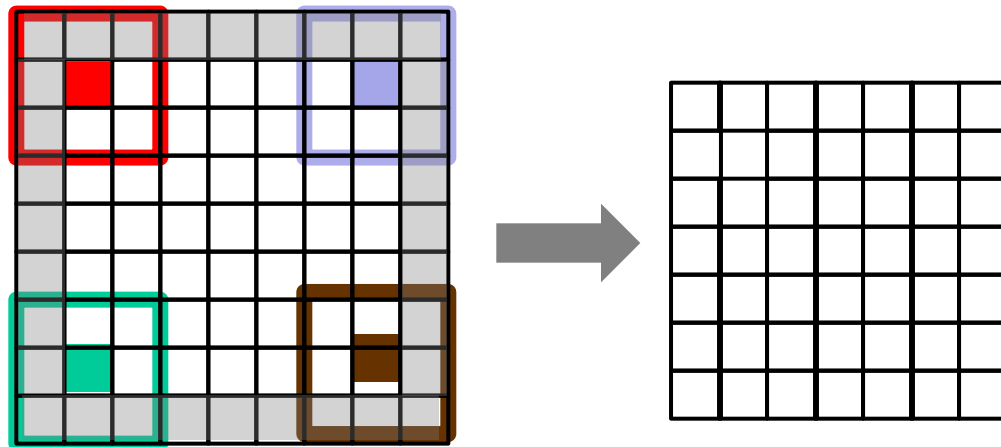
output

Convolutional Layer - Size Change

Output is usually slightly smaller because the borders of the image are left out

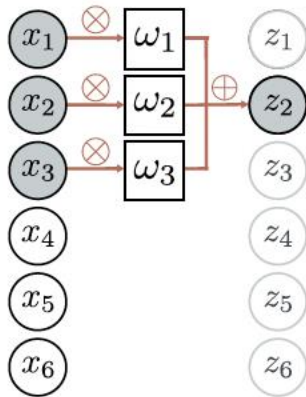


If want output to be the same size, zero-pad the input

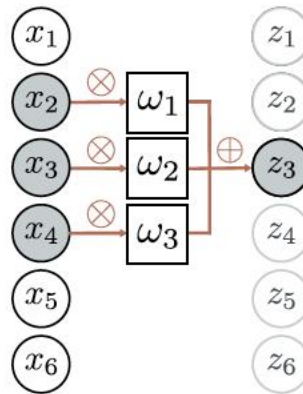


1D Convolution

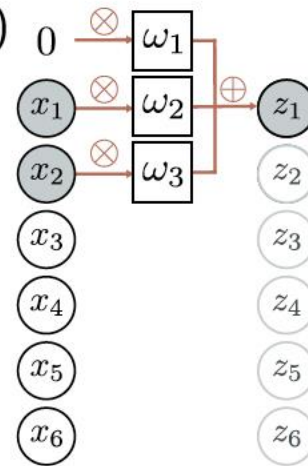
a)



b)



c)



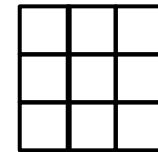
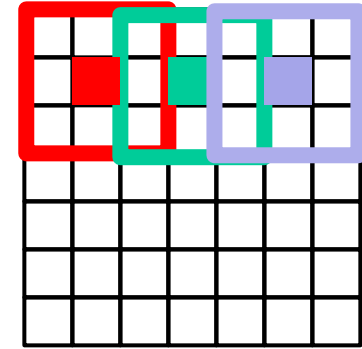
Convolutional Layer - Stride

Can apply convolution only to some pixels (say every second)

- output layer is smaller

Example

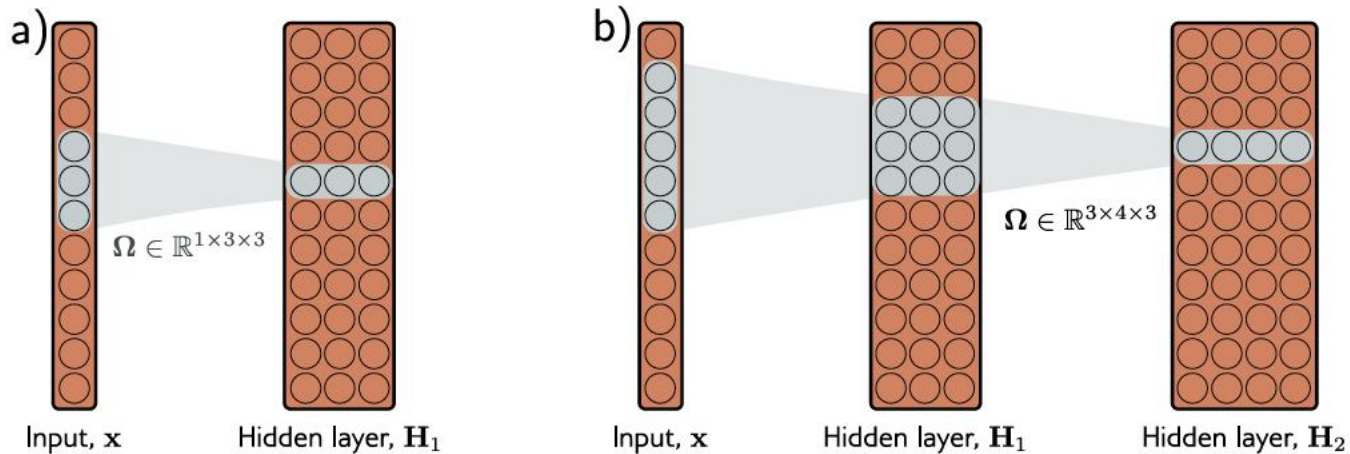
- stride = 2 means apply convolution every second pixel
- makes output image approximately **twice smaller** in each dimension
 - image not zero-padded in this example



strided convolution

reduces spatial resolution of the output

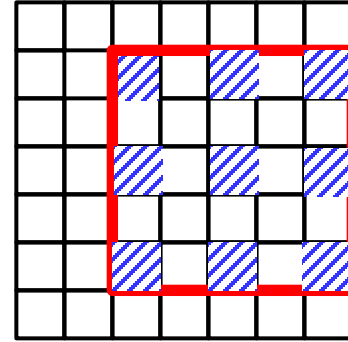
Receptive Field of Convolutional Layer



Convolutional Layer - Dilation

It may be helpful to **increase kernel size**
to **enlarge “receptive field”**
for each element of the output

But larger kernels could be expensive...



Use only **subset of points** within the kernel's window

atrous convolution

(Fr. *à trous* – hole)

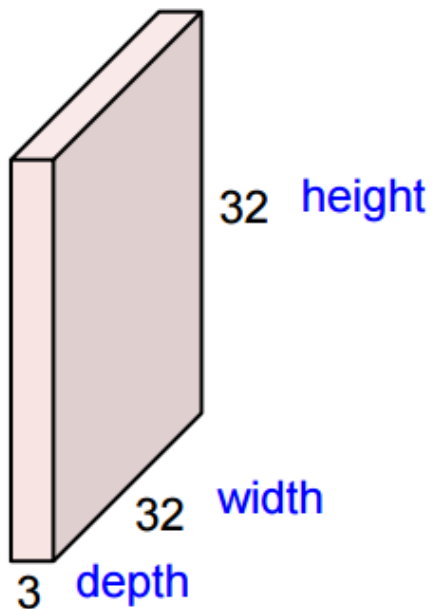
a.k.a. ***dilated convolution***

larger *receptive field* (5x5) for output elements
while effectively using smaller kernels (3x3)

Convolutional Layer – Feature Depth

Input image is usually color, has 3 channels or depth 3

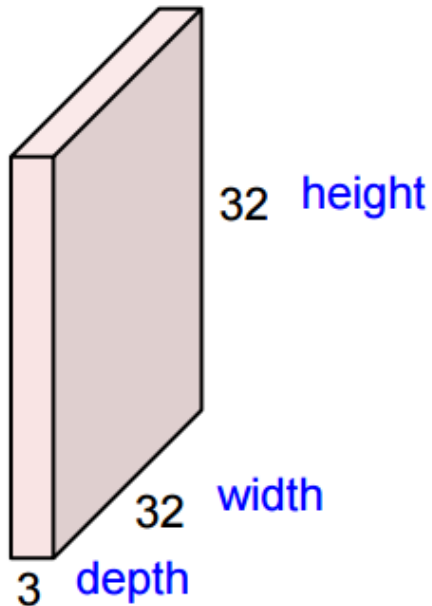
32x32x3 image



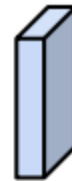
Convolutional Layer – Feature Depth

Convolve 3D image with 3D filter

32x32x3 image



5x5x3 filter



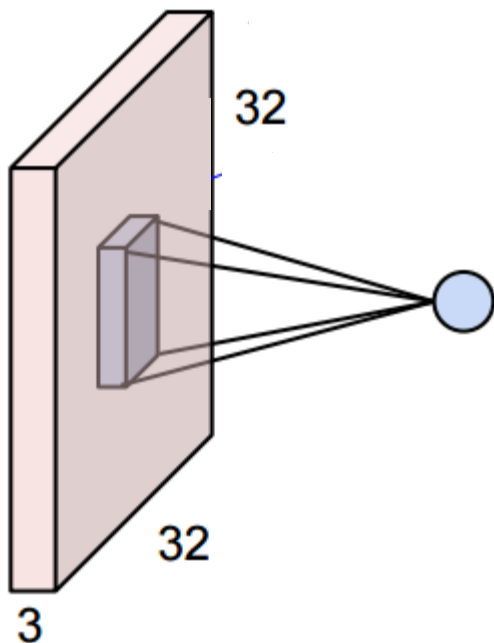
75 parameters

Convolutional Layer – Feature Depth

Each convolution step is a 75 dimensional dot product between the $5 \times 5 \times 3$ filter and a piece of image of size $5 \times 5 \times 3$

Can be expressed as $\mathbf{w}^t \mathbf{x}$, 75 parameters to learn ($\mathbf{w}$)

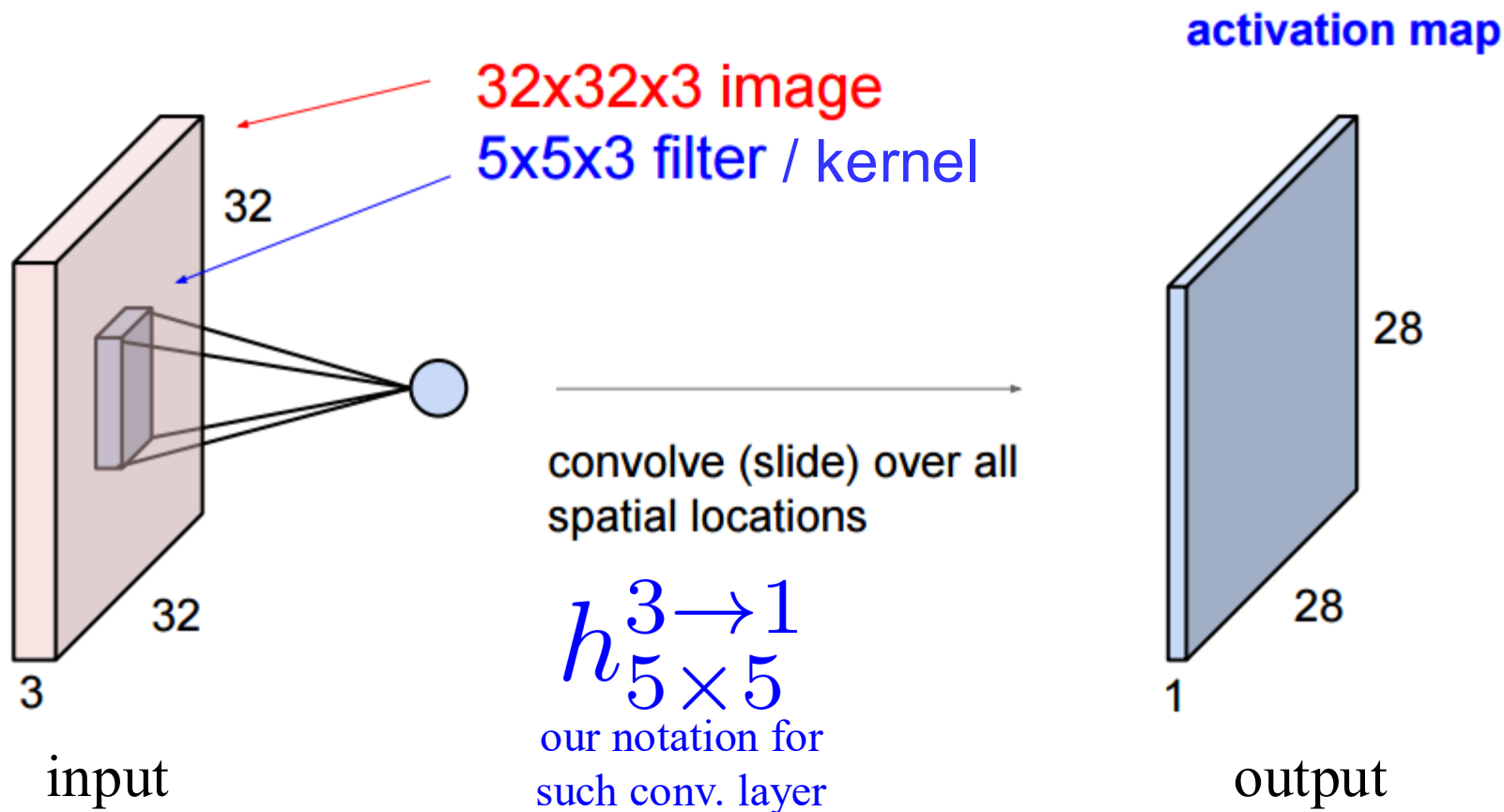
Can add bias $\mathbf{w}^t \mathbf{x} + b$, 76 parameters to learn ($\mathbf{w}, b$)



Convolutional Layer

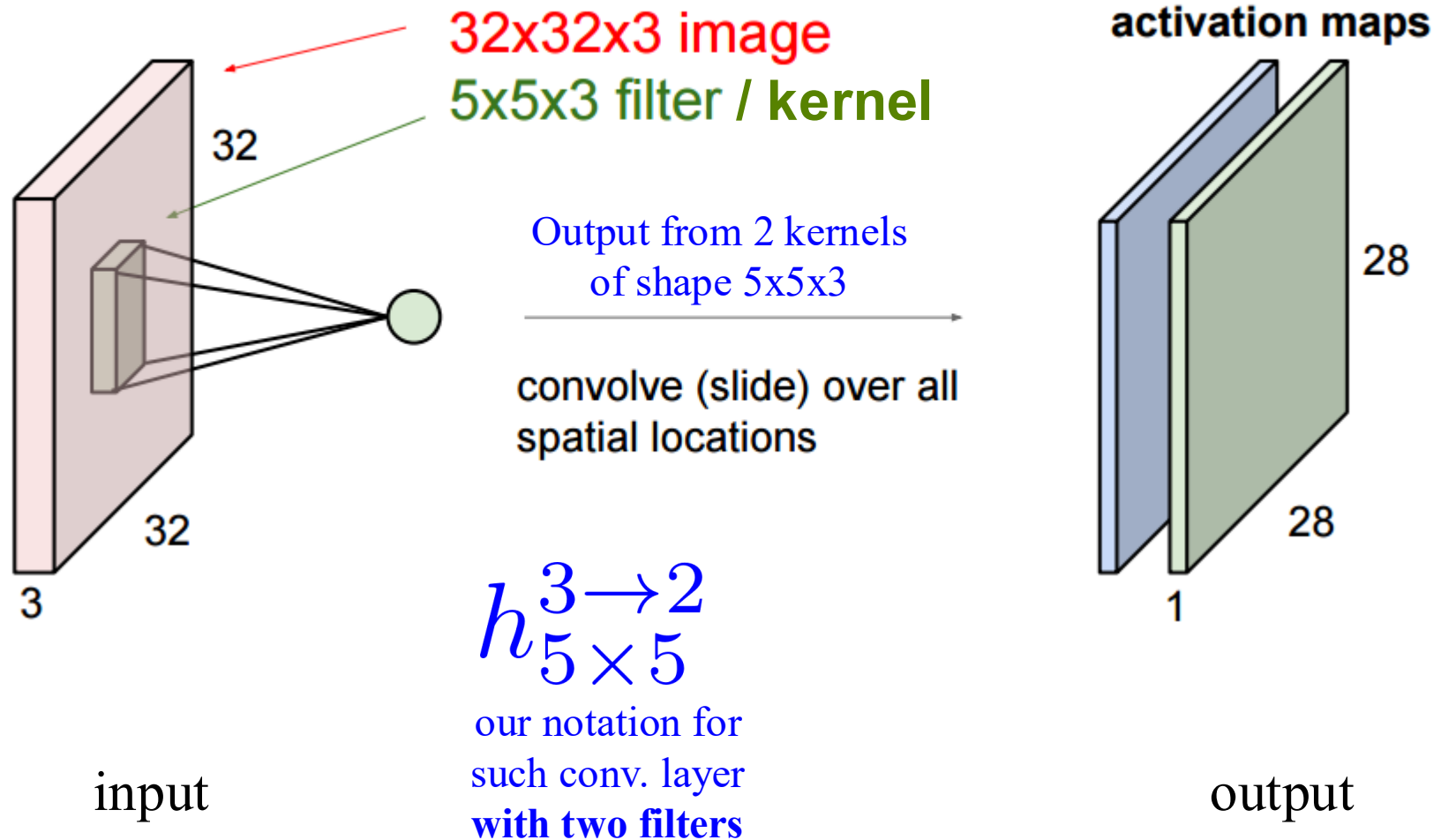
Convolve 3D image with 3D filter

- result is a 28x28x1 activation map, no zero padding used



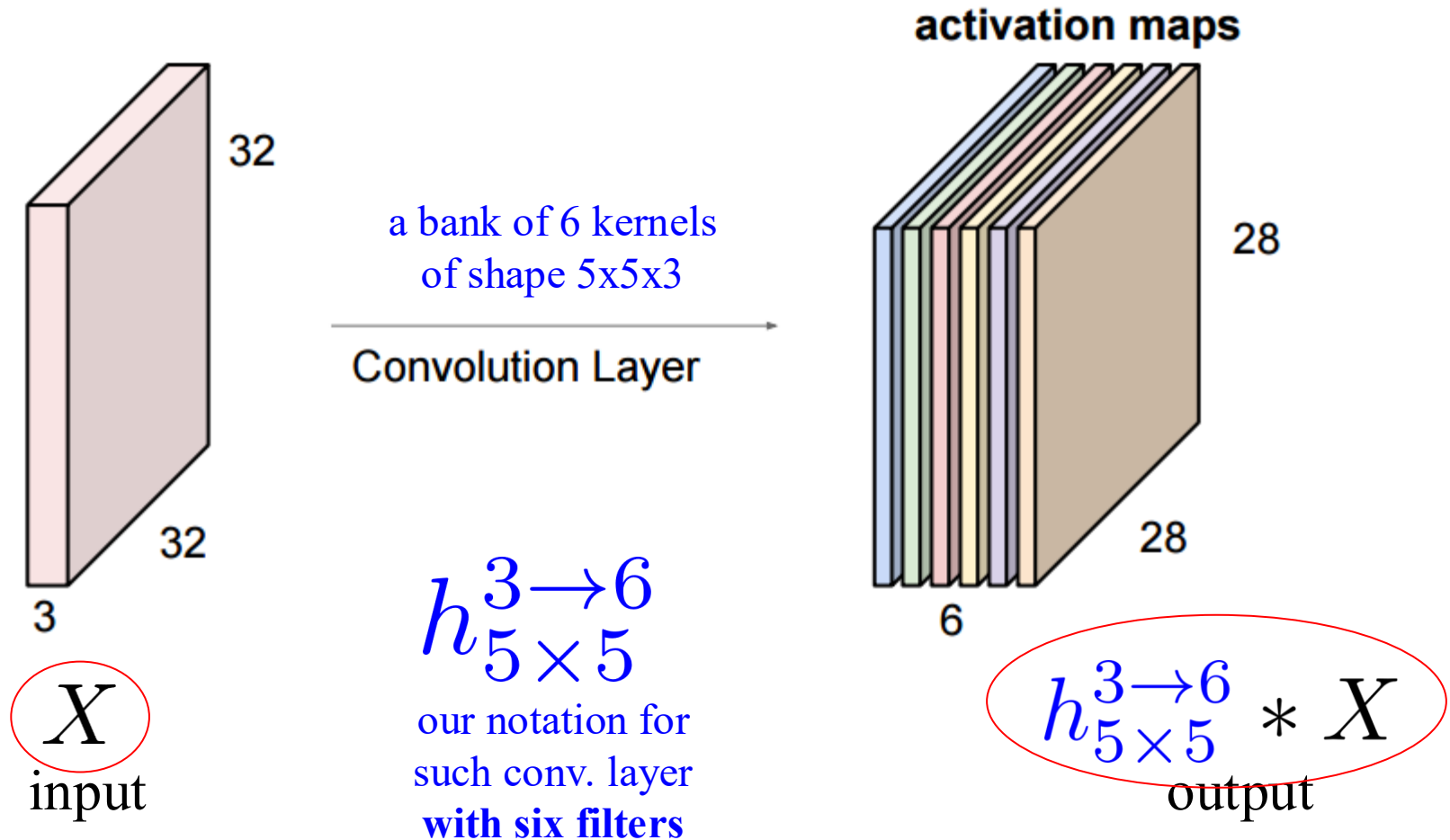
Convolutional Layer

Consider one **extra filter**



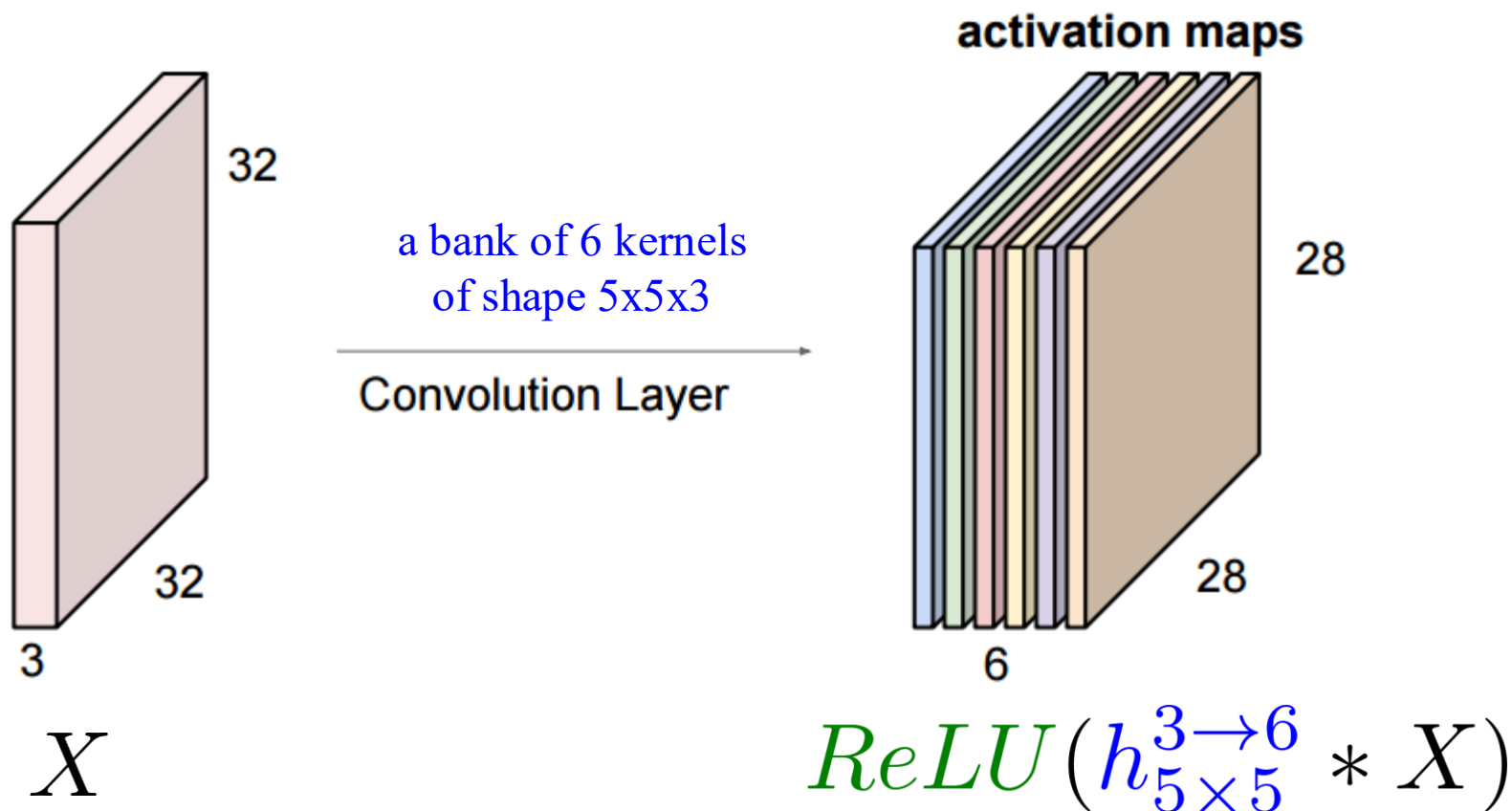
Convolutional Layer

- If have 6 filters (each of size 5x5x3) get 6 activation maps, 28x28 each
- Stack them to get new 28x28x6 “image”



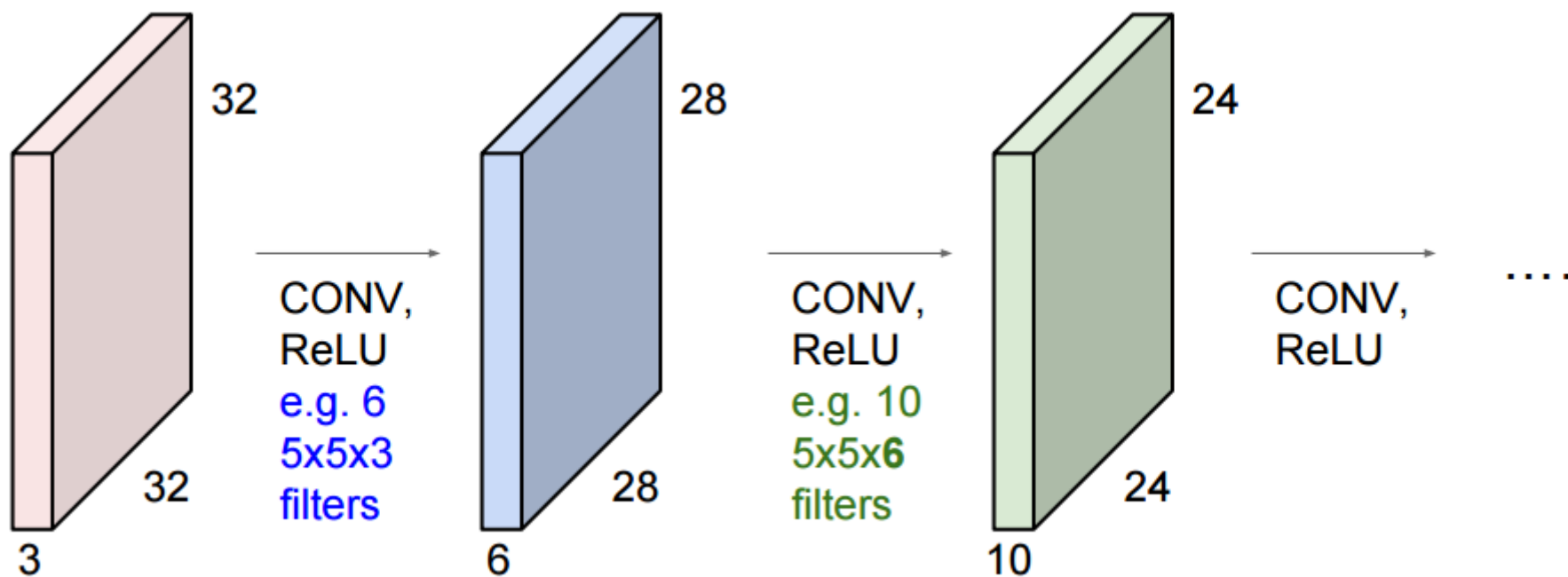
Convolutional Layer

Apply activation function (say **ReLU**) to the activation map



Several Convolution Layers

Construct a sequence of convolution layers interspersed with activation functions



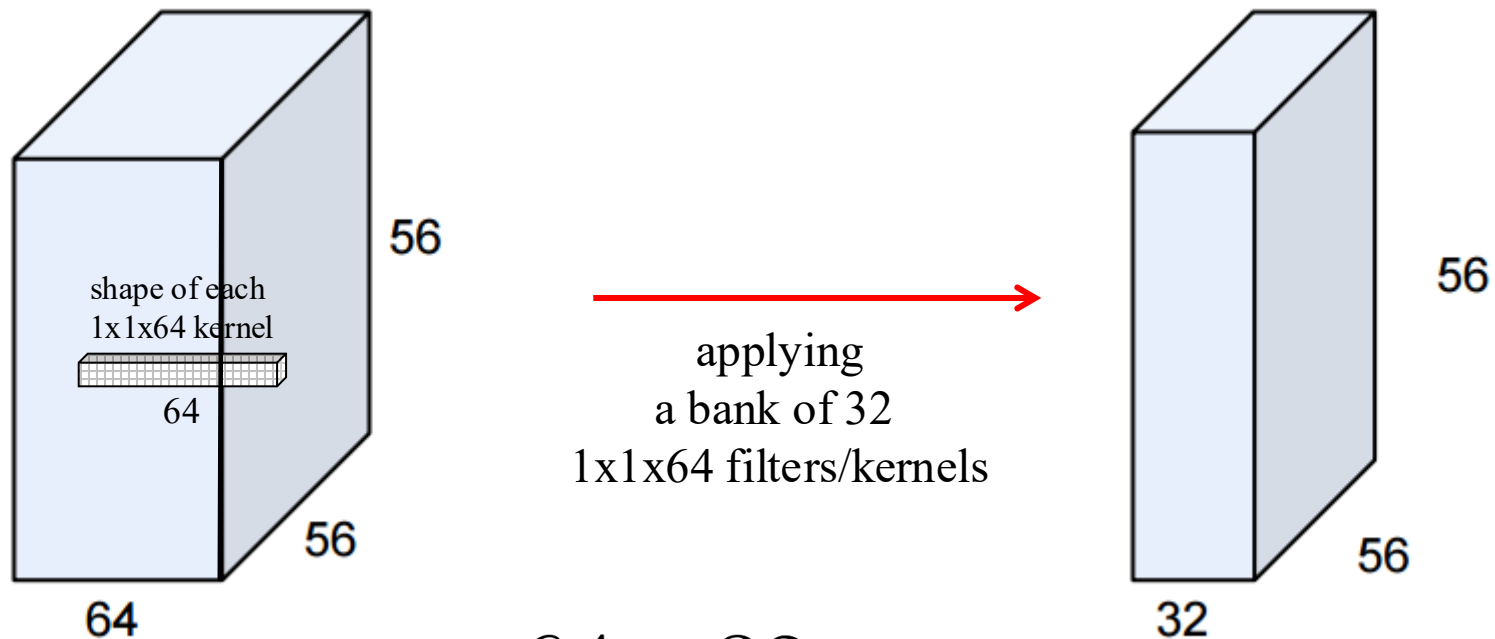
$$ReLU(h_{5 \times 5}^{3 \rightarrow 6} * X) \quad ReLU(h_{5 \times 5}^{6 \rightarrow 10} * X)$$

Convolutional Layer

1x1 convolutions make perfect sense

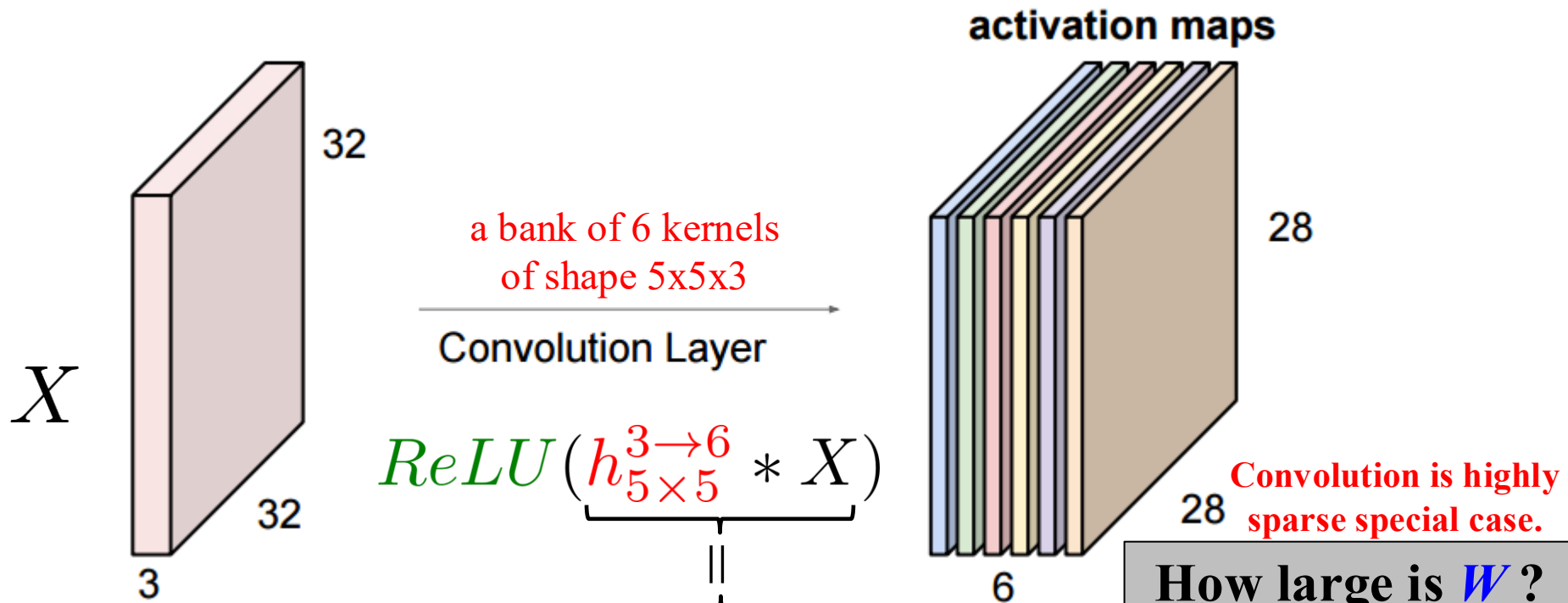
Example

- Input image of size 56x56x64
- Convolve with 32 filters, each of size 1x1x64



$$h_{1 \times 1}^{64 \rightarrow 32} * X$$

Convolutional Layer vs Fully Connected



How large is W ?

$$m = 32 \times 32 \times 3$$

$$n = 28 \times 28 \times 6$$

1.4m parameters

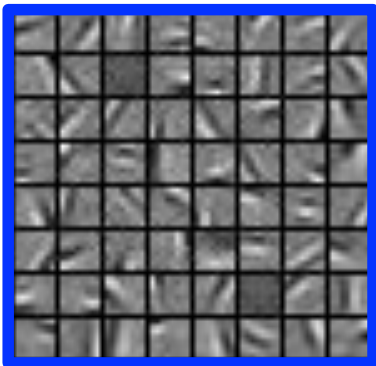
vs. 450 parameters
for 6 kernels 5x5x3

The **convolution** is a linear transform.
So, we can equivalently express it via
matrix multiplication for some matrix W .

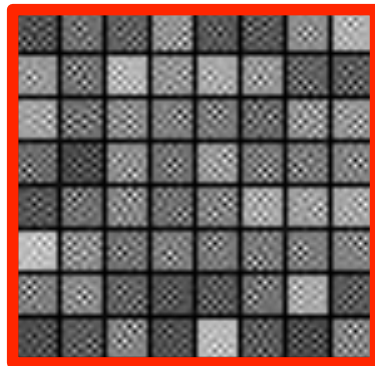
Check Learned Convolutions

- Good training: learned filters exhibit structure and are uncorrelated

GOOD

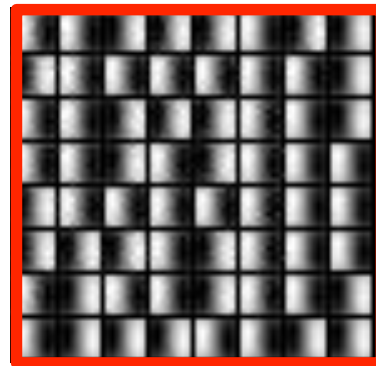


BAD



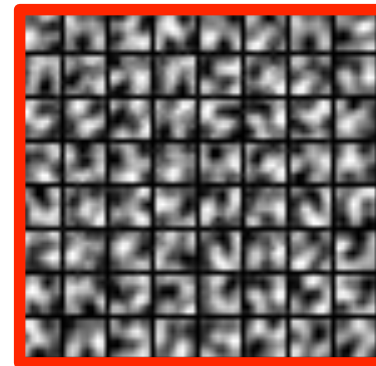
too noisy

BAD



too
correlated

BAD



lack
structure

Convolutional Layer Summary

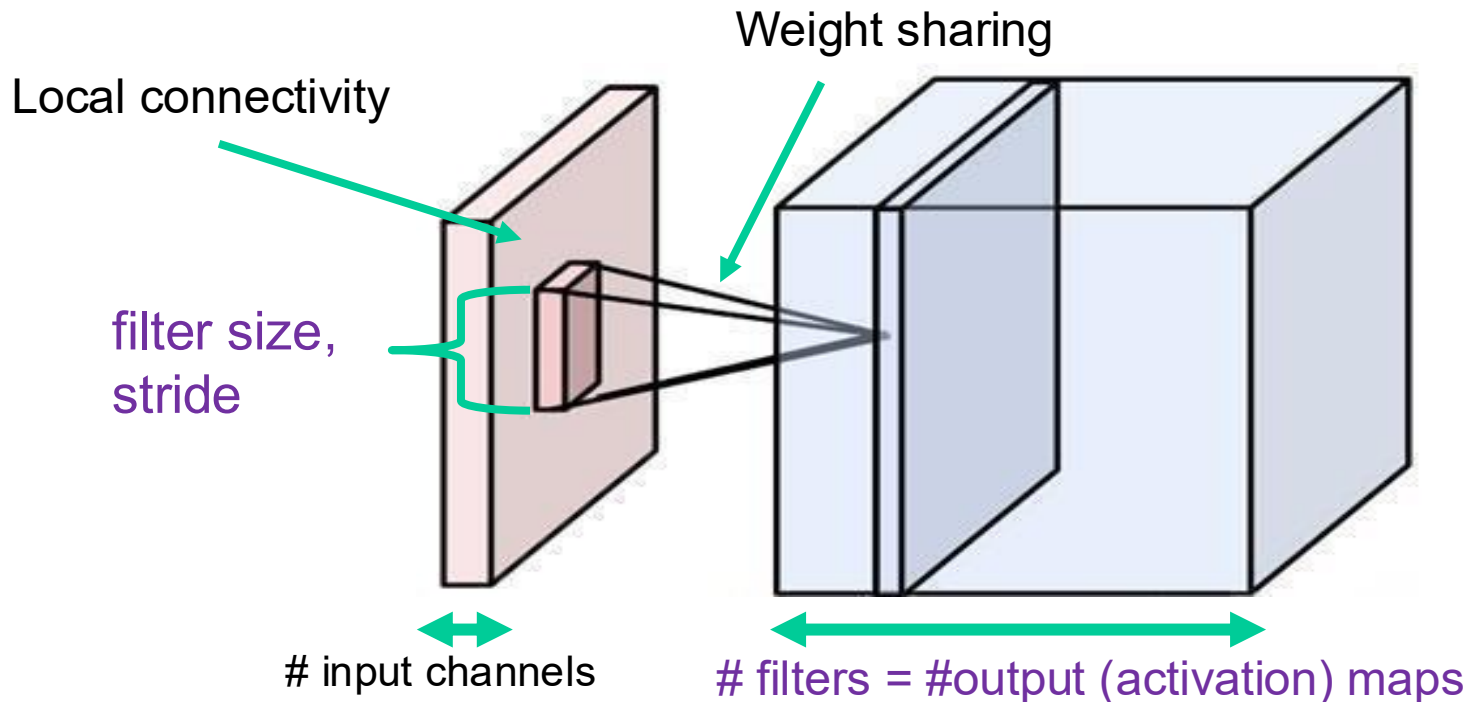
Local connectivity

Weight sharing

Handling multiple input/output channels

Retains location associations

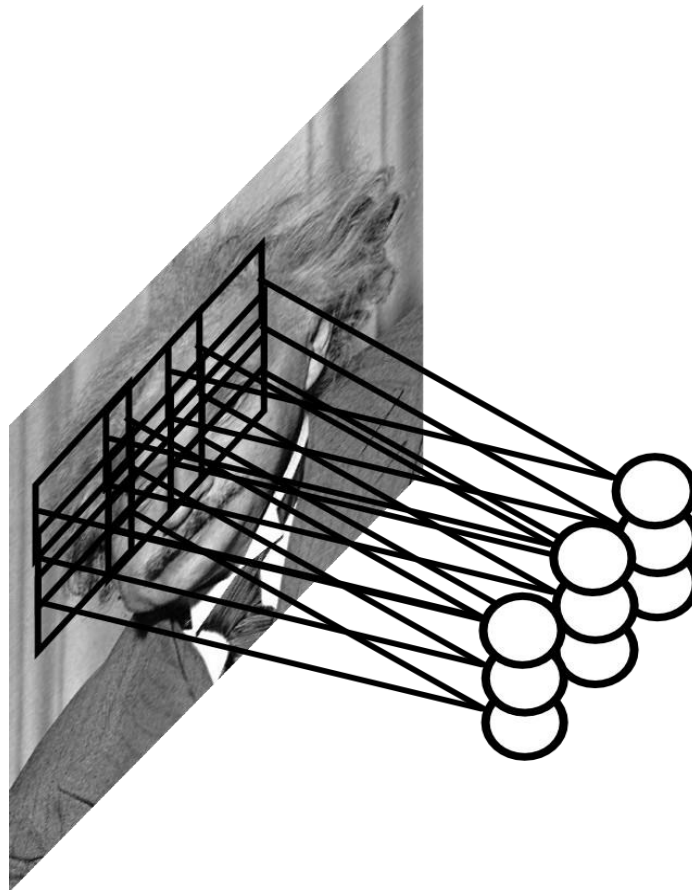
Transforms 3D tensor into 3D tensor (**tensor flow**)



Pooling Layer

Say a filter is an *eye* detector

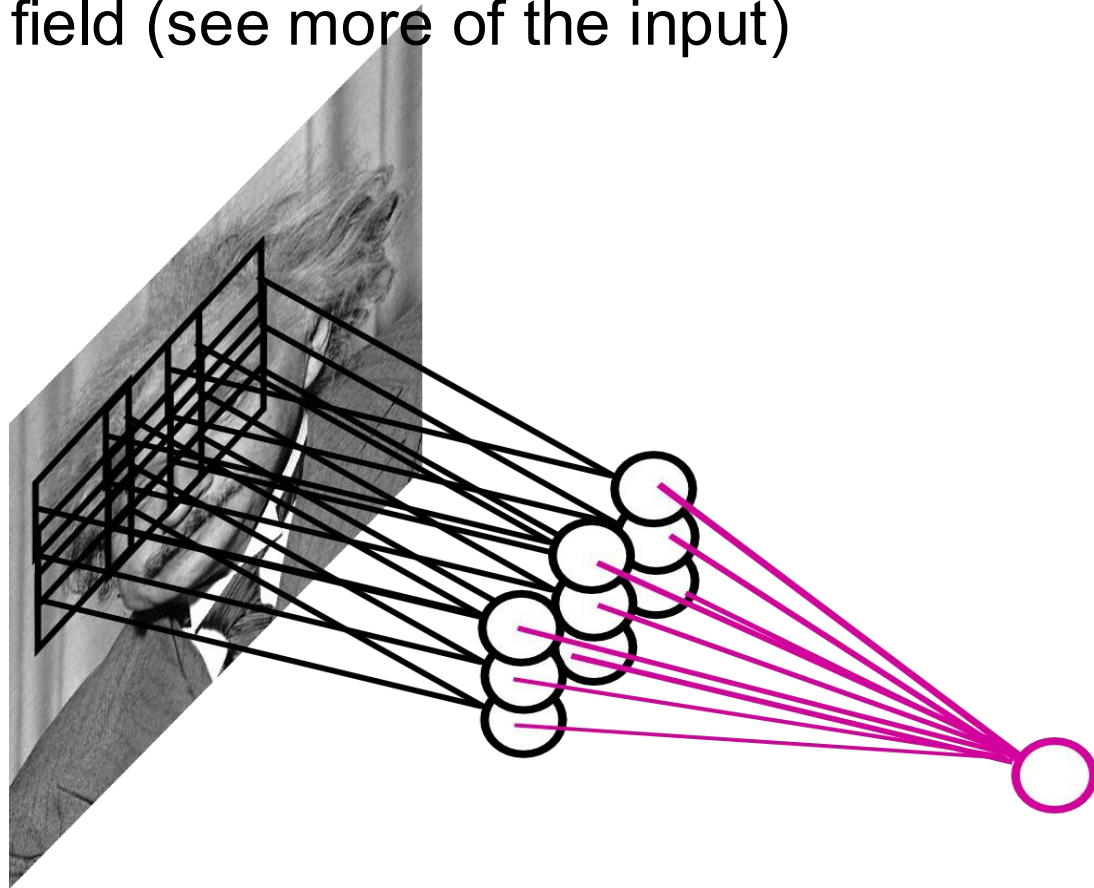
Want detection to be robust to precise *eye* location



Pooling Layer

Pool responses at different locations

- by taking **max**, **average**, etc.
- robustness to exact spatial location
- also larger receptive field (see more of the input)
- Usually pooling applied with stride > 1
- This reduces resolution of output map



Pooling Layer: Max Pooling Example

Single depth slice

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

max pool with 2x2 filters
and stride 2



6	8
3	4

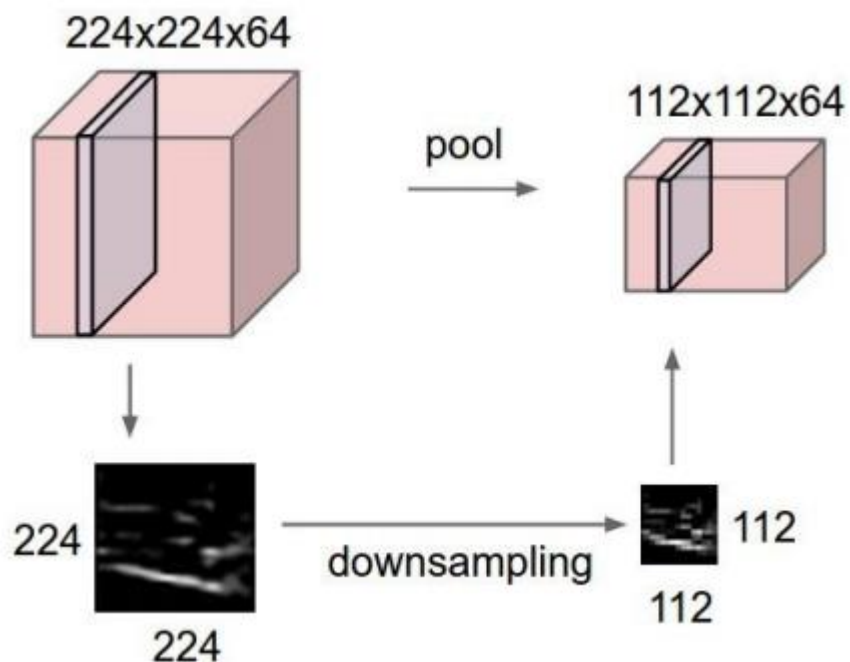
$$\text{Pool}_{2 \times 2}^{st2}(X)$$

our notation for
2 by 2 pooling layer
with stride 2

- pooling can be interpreted as *downsampling*

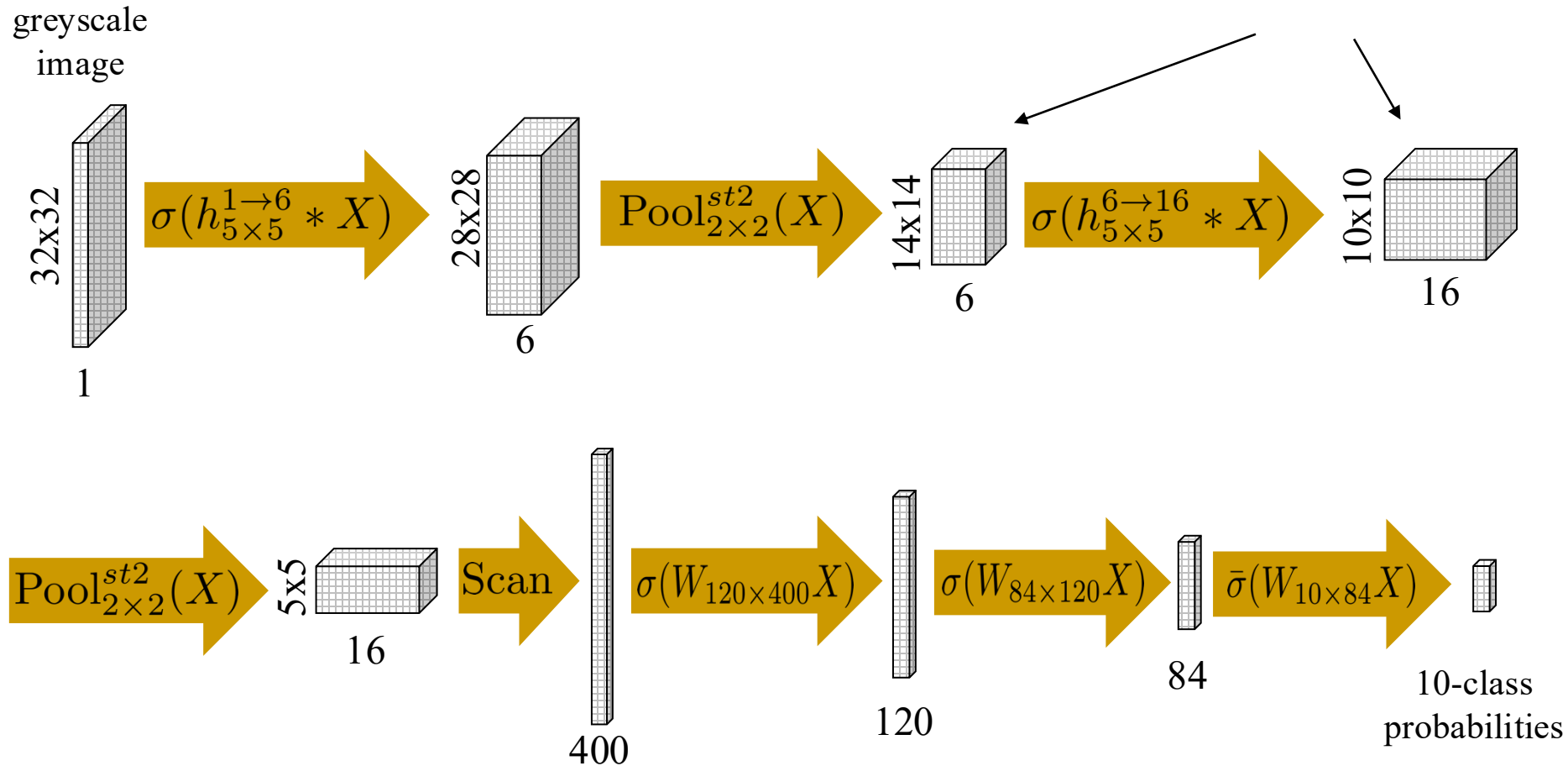
Pooling Layer

Pooling usually applied to each activation map separately



Basic CNN example (à la **LeNet** -1998)

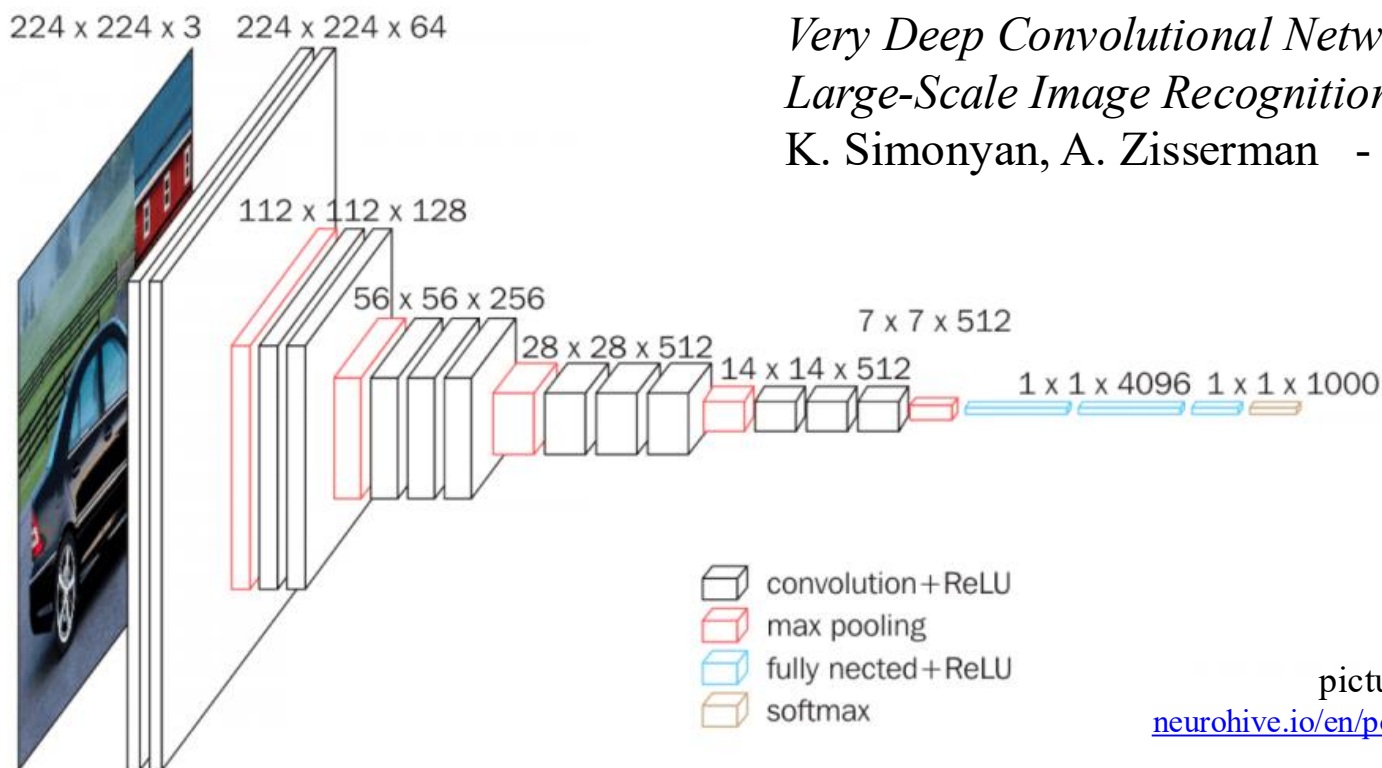
NOTE: transformation of multi-dimensional arrays (**tensors**)



VGG -16

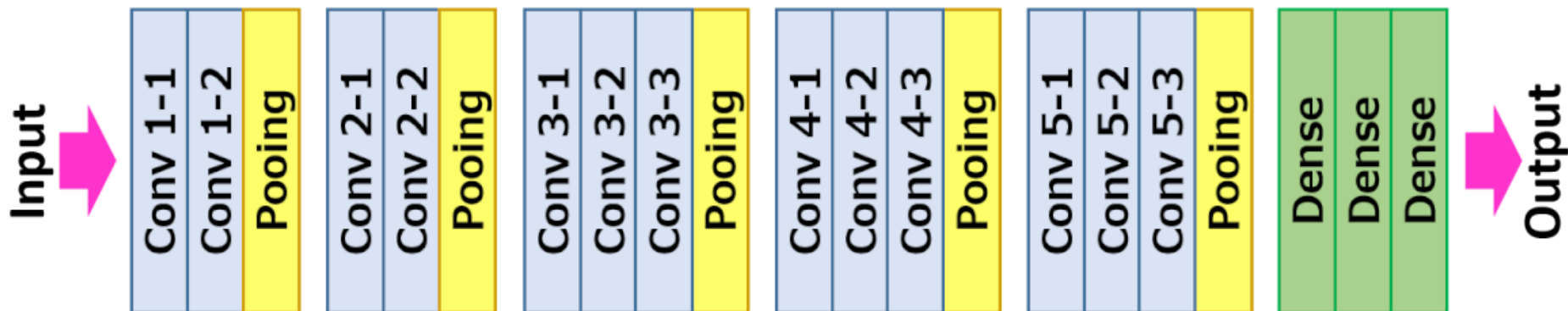
Very Deep Convolutional Networks for Large-Scale Image Recognition

K. Simonyan, A. Zisserman - ICLR 2015



picture credits

neurohive.io/en/popular-networks/vgg16/



Deeper the better?

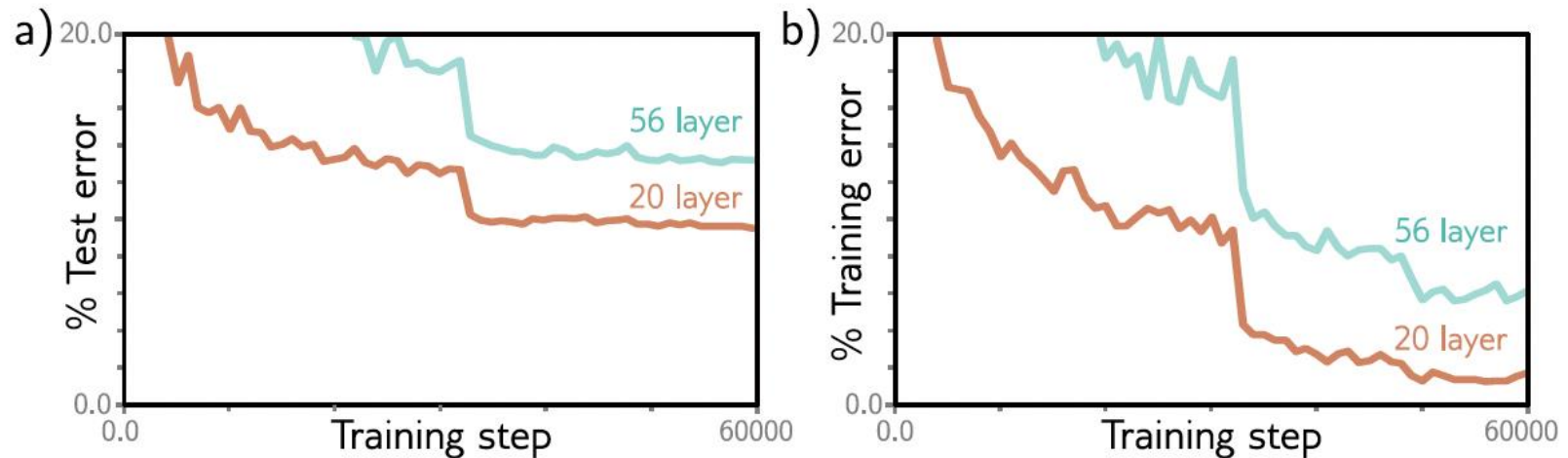


Figure 11.2 Decrease in performance when adding more convolutional layers. a) A 20-layer convolutional network outperforms a 56-layer neural network for image classification on the test set of the CIFAR-10 dataset (Krizhevsky & Hinton, 2009). b) This is also true for the training set, which suggests that the problem relates to training the original network rather than a failure to generalize to new data. Adapted from He et al. (2016a).

Sequential Processing

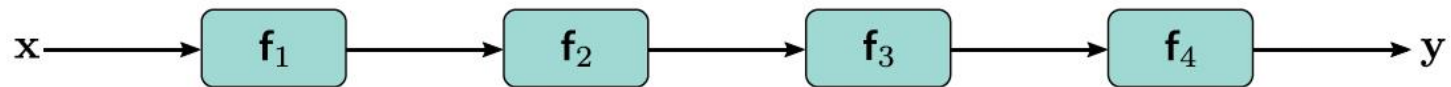
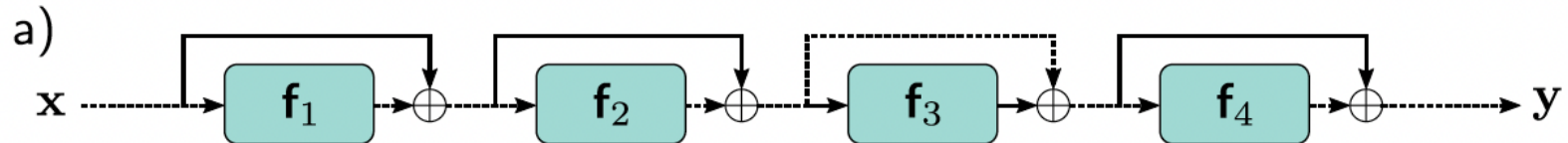


Figure 11.1 Sequential processing. Standard neural networks pass the output of each layer directly into the next layer.

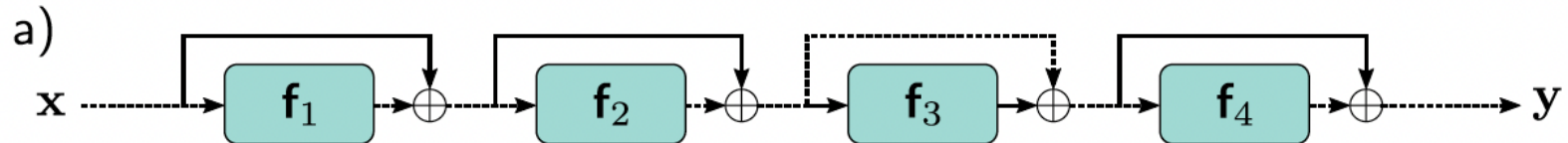
$$y = f_4 \left[f_3 \left[f_2 \left[f_1 [x, \phi_1], \phi_2 \right], \phi_3 \right], \phi_4 \right].$$

Residual Connection



$$\begin{aligned} y = & x + f_1[x] \\ & + f_2[x + f_1[x]] \\ & + f_3[x + f_1[x] + f_2[x + f_1[x]]] \\ & + f_4[x + f_1[x] + f_2[x + f_1[x]] + f_3[x + f_1[x] + f_2[x + f_1[x]]]], \end{aligned} \quad (11.5)$$

Gradient for Residual Connection



$$\frac{\partial y}{\partial f_1} = \mathbf{I} + \frac{\partial f_2}{\partial f_1} + \left(\frac{\partial f_3}{\partial f_1} + \frac{\partial f_2}{\partial f_1} \frac{\partial f_3}{\partial f_2} \right) + \left(\frac{\partial f_4}{\partial f_1} + \frac{\partial f_2}{\partial f_1} \frac{\partial f_4}{\partial f_2} + \frac{\partial f_3}{\partial f_1} \frac{\partial f_4}{\partial f_3} + \frac{\partial f_2}{\partial f_1} \frac{\partial f_3}{\partial f_2} \frac{\partial f_4}{\partial f_3} \right), \quad (11.6)$$

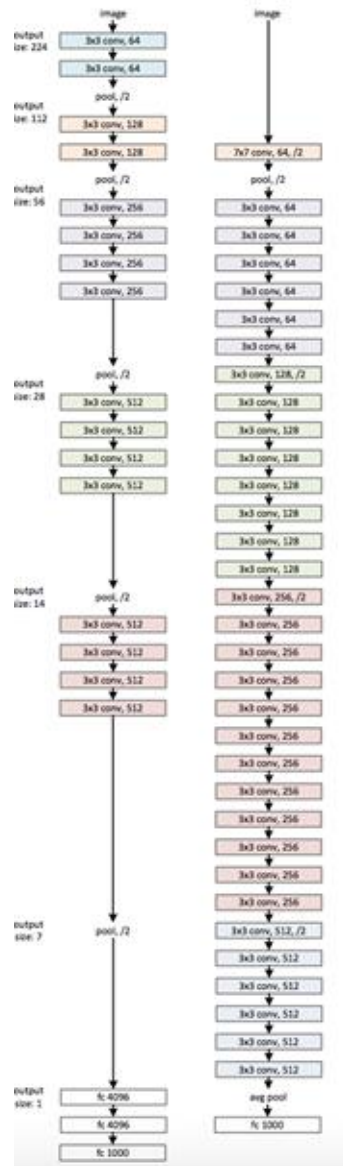
ResNet

very deep ☺

one of the
state of the art
on *image net*

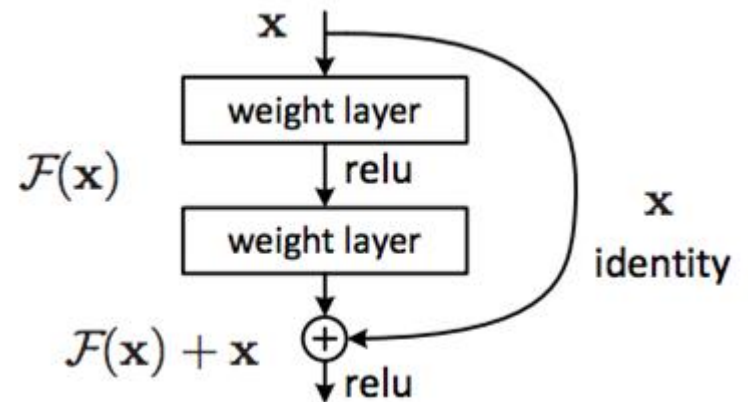
www.image-net.org

- very large dataset of labeled images
 >14,000,000



Deep residual learning for image recognition. K. He, X. Zhang, S. Ren, and J. Sun. CVPR 2016

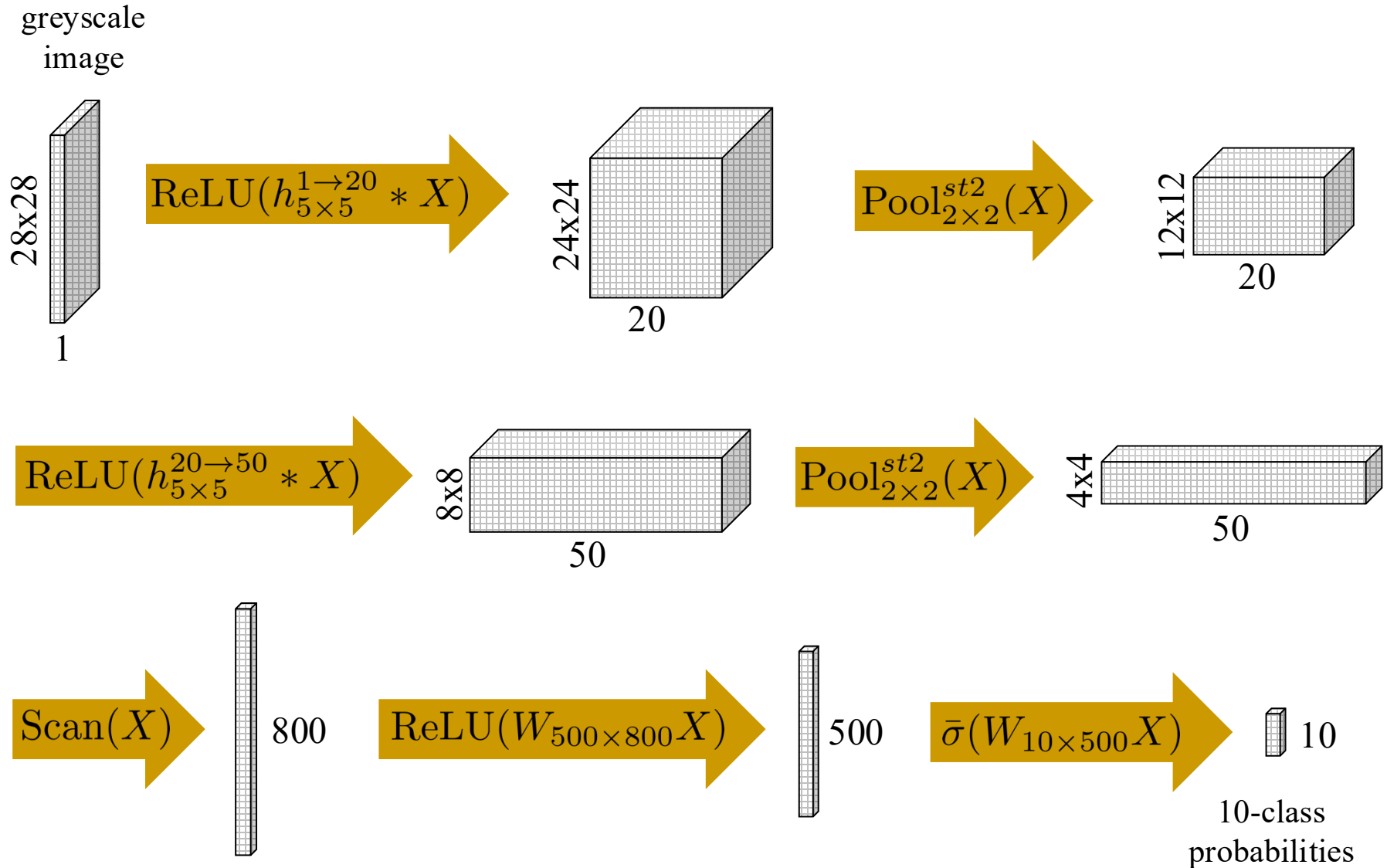
key technical trick



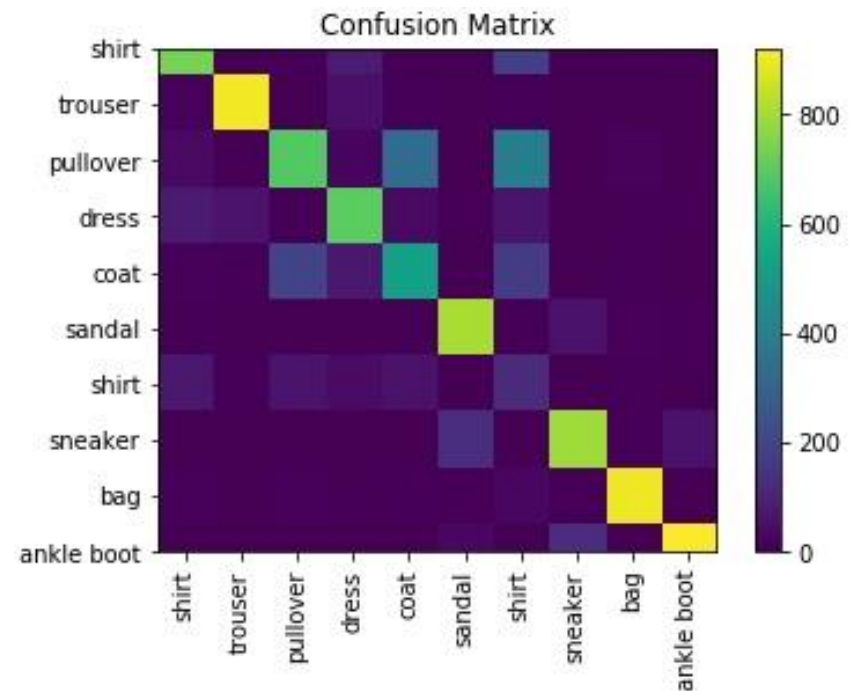
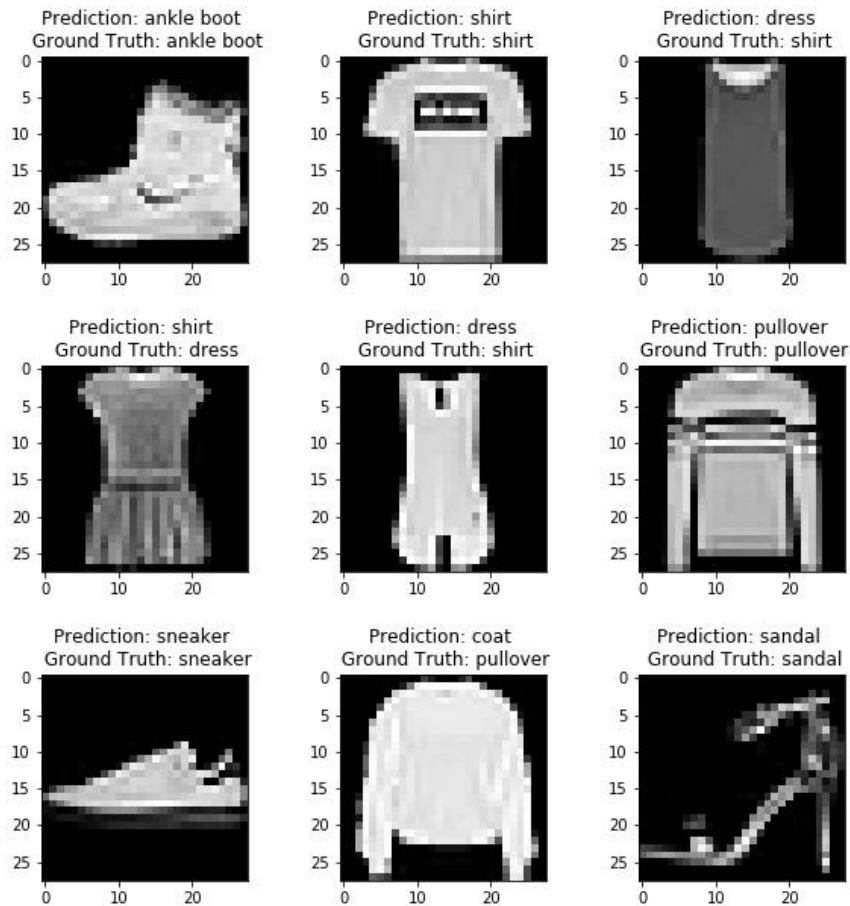
resnet block

(residual link helps gradient descent)

FashionMNIST classification example

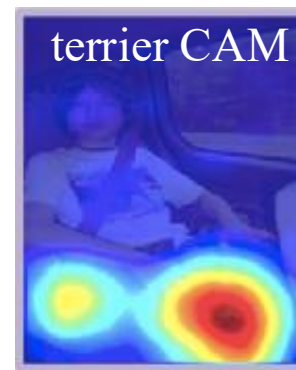


FashionMNIST classification example



Class-activation Map (CAM)

How to localize object given a pretrained image classifier?



CVPR 2016: “Learning Deep Features for Discriminative Localization”
B.Zhou, A.Khosla, A. Lapedriza, A.Oliva, A.Torralba

Class-activation Map (CAM)

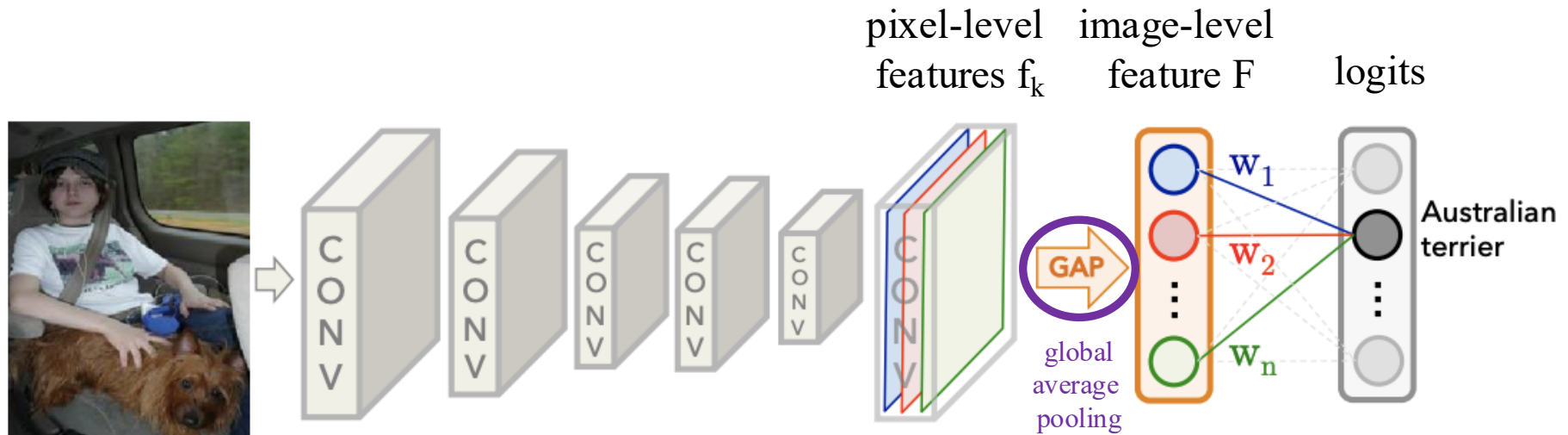
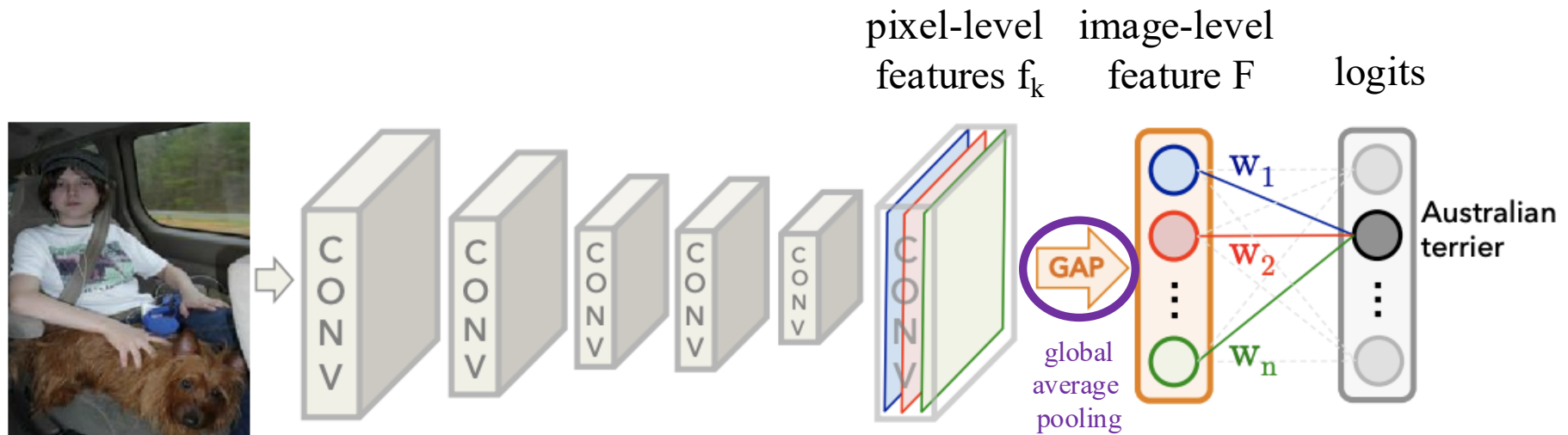


Image $\rightarrow$ Conv layers $\rightarrow$ Global Average Pooling (GAP) $\rightarrow$ Linear layer $\rightarrow$ Softmax

Class-activation Map (CAM)



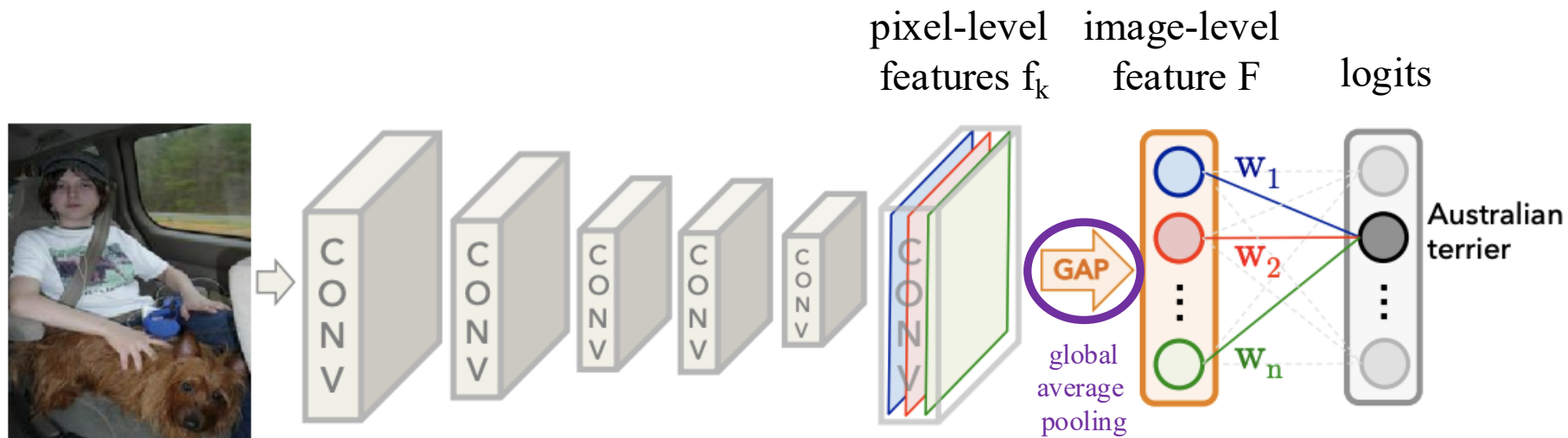
- $f_k(x, y)$ = activation of feature map k at spatial location (x, y)
- There are K feature maps
- GAP computes:

$$F_k = \sum_{x,y} f_k(x, y)$$

Then class score for class c :

$$S_c = \sum_k w_k^c F_k$$

Class-activation Map (CAM)



Substitute:

$$S_c = \sum_k w_k^c \sum_{x,y} f_k(x,y)$$

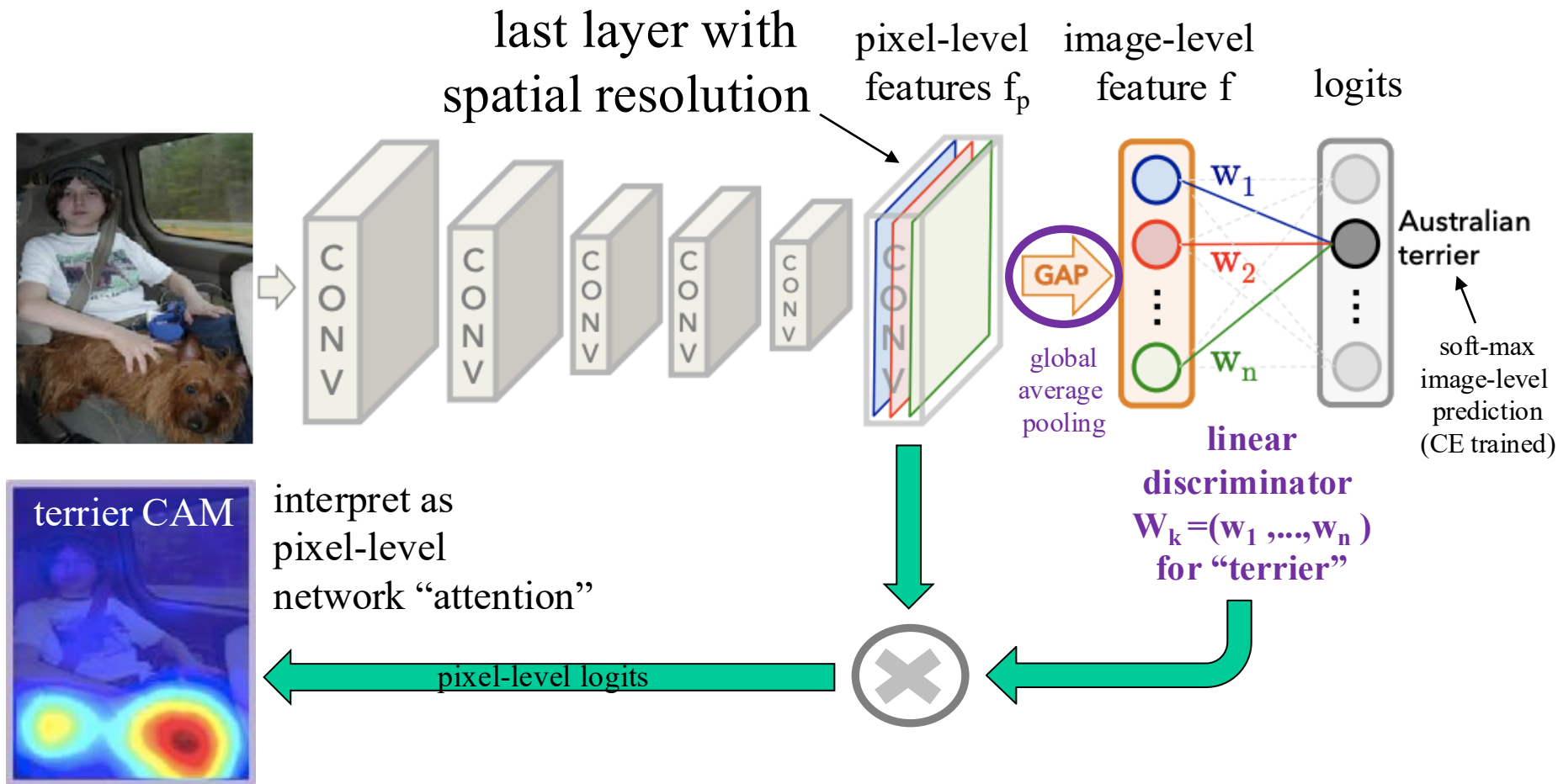
Rearrange:

$$S_c = \sum_{x,y} \sum_k w_k^c f_k(x,y)$$

Define:

$$M_c(x,y) = \sum_k w_k^c f_k(x,y)$$

Class-activation Map (CAM)



Quiz

Fill in the shape of the output volume and the number of parameters at each layer. You can write the activation shapes in the format (H, W, C) for 3D volume, where H, W, C are the height, width and channel dimensions, respectively. Unless specified, assume padding 1, stride 1 where appropriate.

Notation:

- CONVx-N denotes a convolutional layer with N filters with kernel height and width x
- POOL-N denotes a N×N max-pooling layer with stride of N and no padding.
- FLATTEN flattens its inputs to 1d
- FC-N denotes a fully-connected layer with N neurons

Layer	Activation volume dimensions	Number of parameters (weights and biases)
Input	(32, 32, 3)	0
Conv3-8		
ReLU		
POOL-2		
BATCHNORM		
Conv3-16		
ReLU		
POOL-2		
FLATTEN		
FC-10		

Quiz

Fill in the shape of the output volume and the number of parameters at each layer. You can write the activation shapes in the format (H, W, C) for 3D volume, where H, W, C are the height, width and channel dimensions, respectively. Unless specified, assume padding 1, stride 1 where appropriate.

Notation:

- CONVx-N denotes a convolutional layer with N filters with kernel height and width x
- POOL-N denotes a N×N max-pooling layer with stride of N and no padding.
- FLATTEN flattens its inputs to 1d
- FC-N denotes a fully-connected layer with N neurons

Layer	Activation volume dimensions	Number of parameters (weights and biases)
Input	(32, 32, 3)	0
Conv3-8	(32, 32, 8)	$(3 \times 3 \times 3 + 1) \times 8 = 224$
ReLU	(32, 32, 8)	0
POOL-2	(16, 16, 8)	0
BATCHNORM	(16, 16, 8)	$2 \times 8 = 16$
Conv3-16	(16, 16, 16)	$(3 \times 3 \times 8 + 1) \times 16 = 1168$
ReLU	(16, 16, 16)	0
POOL-2	(8, 8, 16)	0
FLATTEN	(1, 1, 1024)	0
FC-10	(1, 1, 10)	$(1024 + 1) \times 10 = 10250$